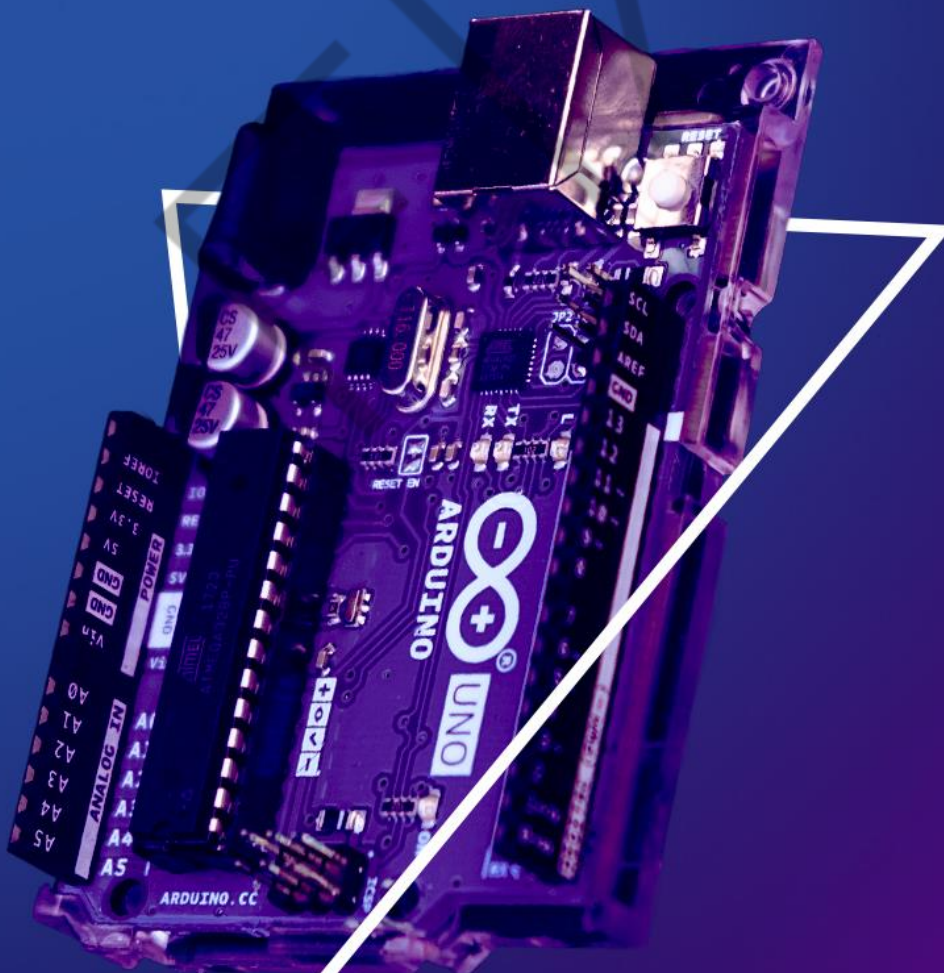


HACKING THE PLANET

ARDUINO



5

LISTA DE FIGURAS

Figura 1 – Arduino Uno	6
Figura 2 – Aplicação simples em jardins	7
Figura 3 – Representação de um sistema de alarme	7
Figura 4 – Modelo Raspberry-Pi B+	8
Figura 5 – Plataforma online Tinker Cad	9
Figura 6 – Plataforma online Tinkercad	10
Figura 7 – Passos para a criação de conta	10
Figura 8 – Passos para a criação de conta	11
Figura 9 – Criação de novos circuitos	12
Figura 10 – Área Aprenda do Tinkercad	13
Figura 11 – Circuito Elétrico X Hidráulico	14
Figura 12 – Exemplo de Protoboard	15
Figura 13 – Controle de luz com potenciômetro	17
Figura 14 – Simulação do Controle de luz com potenciômetro	20
Figura 15 – Controle de Semáforo	21
Figura 16 – Alarme com sensor ultrassônico	24
Figura 17 – Programação por blocos	25
Figura 18 – Simulação do alarme com sensor ultrassônico	26
Figura 19 – Inclusão do Gateway IoT	34
Figura 20 – Tela inicial do Arduino IDE	35
Figura 21 – Diagrama para o pisca led	36
Figura 22 – Circuito real para o pisca led	37
Figura 23 – Código para o pisca led	37
Figura 24 – Escolha da placa e porta	38
Figura 25 – Acesso ao monitor serial	40
Figura 26 – Acionamento do led via serial	42
Figura 27 – Linha de comando do Node.js	44
Figura 28 – Subindo o serviço do Node-Red	45
Figura 29 – Tela inicial do Node-Red	46
Figura 30 – Configuração da porta serial no Node-Red	48
Figura 31 – Exemplo de fluxo usando Node-Red	49
Figura 32 – Detalhamento do nó inject do Node-Red	49
Figura 33 – Recebimento de dados da placa via Node-Red	50
Figura 34 – Subindo localmente o MQTT broker Mosquitto	50
Figura 35 – Fluxo Node-Red mostrando os tópicos	51
Figura 36 – Configuração dos nodes MQTT via Node-Red	51
Figura 37 – Fluxo para teste do Gateway via Node-Red	51
Figura 38 – Aba “debug” do editor de fluxo do Node-Red	52

LISTA DE TABELAS

Tabela 1 – Componentes do Projeto 1.....	18
Tabela 2 – Componentes do Projeto 2.....	23
Tabela 3 – Componentes do Projeto 3.....	27

EMANIP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Exemplo de código-fonte Arduino	19
Código-fonte 2 – Controle de Luz com Potenciômetro	19
Código-fonte 3 – Controle de semáforo	22
Código-fonte 4 – Alarme com sensor ultrassônico	26
Código-fonte 5 – Temperatura e Umidade	39
Código-fonte 6 – Pisca Led	41
Código-fonte 7 – Uso de JSON via porta Serial	43
Código-fonte 8 – Controle de led e luminosidade via Node-Red.....	48

SUMÁRIO

ARDUINO, O PEQUENO NOTÁVEL	6
1 INTRODUÇÃO	6
2 ALGUMAS APLICAÇÕES COM ARDUINO	7
3 COMPARANDO PLACAS	8
3.1 Arduino versus Raspberry-pi	8
4 ALGUNS PRÉ-REQUISITOS	9
5 EFETUANDO O CADASTRO NO TINKERCAD	9
6 TREINAR É PRECISO	12
7 DESENVOLVENDO PROJETOS	13
7.1 Circuitos elétricos	13
8 PLACA PROTOBOARD, MATRIZ DE CONTATO OU BREADBOARD	14
9 OS SENSORES	15
10 DESENVOLVIMENTO DE PROJETOS	16
10.1 Projeto 1 – Controle de Luz com Potenciômetro	16
10.2 Projeto 2 – controle de semáforo	21
10.3 Projeto 3 – Alarme com sensor ultrassônico	23
11 CONSTRUÇÃO DO GATEWAY COM O MQTT	27
11.1 Sobre o MQTT	28
11.2 Sobre o CoAP	31
11.3 Construção do gateway	33
11.4 Arduino IDE	35
11.5 Compilando o primeiro programa	36
11.5.1 Inserindo uma nova biblioteca	38
11.5.2 Enviando os dados via serial	40
11.5.3 Uso de JSON	42
11.6 Apresentação do Gateway	44
11.6.1 Sobre os nós do Node-Red	46
11.6.2 Usando o broker Mosquitto	50
12 CONCATENANDO AS IDEIAS APRESENTADAS	52
REFERÊNCIAS	53

ARDUINO, O PEQUENO NOTÁVEL

1 INTRODUÇÃO

O Arduino foi criado em 2005 por um grupo de pesquisadores italianos do *Interaction Design Institute of Ivrea* pela necessidade de obter um microcontrolador acessível aos alunos para seus projetos de robótica, dada a dificuldade e o alto custo para trabalhar com as soluções existentes até então.

Nada mais é que a junção de um hardware – a placa mãe, um microcontrolador Atmel AVR – aliado a um software IDE (*Integrated Development Environment* ou Ambiente de Desenvolvimento Integrado) implementado para ajudar de forma simples na prototipação eletrônica de sistemas interativos.

Seu funcionamento é como o de um computador que armazena entradas e saídas de dados para criar sistemas automatizados. Baseado todo no conceito *open-source*, as placas Arduino possuem sensores distribuídos em módulos para compra que permitem a customização da solução para cada tipo de projeto.

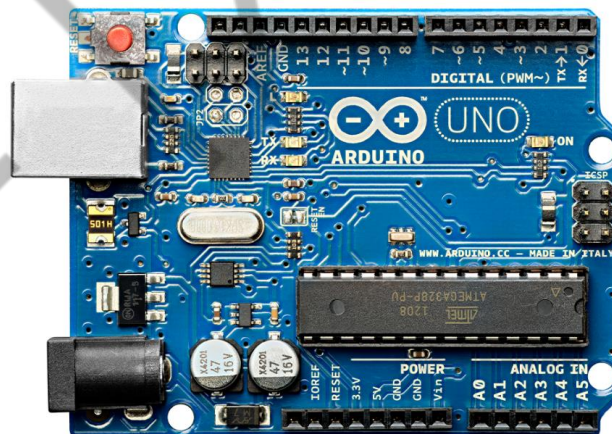


Figura 1 – Arduino Uno
Fonte: Arduino (2020)

Hoje, o Arduino é uma plataforma completa para a construção de um sistema automatizado eletrônico que possui hardware e software integrados e fáceis de usar, que permite que pessoas sem conhecimento técnico consigam criar objetos interativos.

2 ALGUMAS APLICAÇÕES COM ARDUINO

Uma aplicação interessante está no setor de irrigação de plantas acionado por um circuito que mede a temperatura e o grau de intensidade de luz, que passou a ser chamado de Garduino (a junção das palavras Garden e Arduino), e que pode ser visto na figura abaixo.



Figura 2 – Aplicação simples em jardins
Fonte: Garduino (2020)

Agora, veja como seria um sistema futurista de alarme por meio de uso de feixes luminosos, e neste caso é claro que a figura abaixo se trata somente de uma representação, mas seria possível fazer algo mais concreto usando um Arduino e sensores de presença, de movimento ou aproximação. É só deixar a imaginação te levar nas imensas possibilidades de executar o projeto.

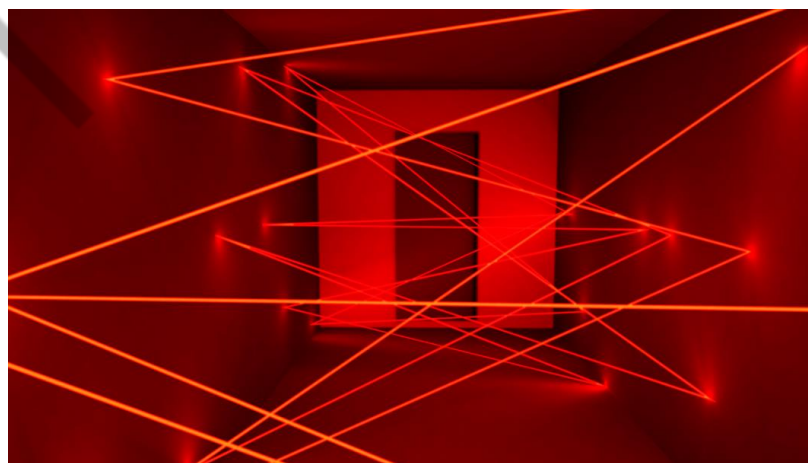


Figura 3 – Representação de um sistema de alarme
Fonte: Farinaro (2020)

3 COMPARANDO PLACAS

3.1 Arduino versus Raspberry-pi

Criado em 2012 pela Raspberry-Pi Foundation para estimular o estudo de Ciências da Computação em escolas, o Raspberry-Pi é um microprocessador para auxiliar no desenvolvimento de softwares, protótipos, experimentos. Sua estrutura é pequena, do tamanho de um cartão de crédito, mas é um computador completo com cabo de rede, áudio, vídeo e cartão micro SD. O caráter aberto da plataforma, também em um conceito de hardware livre, não impõe limites em usos e aplicações.

Para efeito de ilustração, observe abaixo o modelo:



Figura 4 – Modelo Raspberry-Pi B+
Fonte: RASPBERRY (2020)

Ao contrário do Raspberry Pi, as placas do Arduino são microcontroladores, não computadores completos. Eles não executam um sistema operacional, simplesmente executam códigos interpretados pelo seu *firmware*. Você perde o acesso às ferramentas básicas que um sistema operacional fornece, mas por outro lado, a execução direta de código simples é mais fácil e é realizada sem sobrecarga do sistema operacional, uma das vantagens do Arduino.

O principal objetivo da placa Arduino é a interface com sensores e dispositivos, por isso é perfeito para projetos de hardware em que seja necessário que as “coisas” respondam a várias leituras do sensor ou entrada manual. Isso pode não parecer algo substancial, mas na verdade é um sistema muito sofisticado que permite gerenciar melhor seus dispositivos. É também adequado para a interface com outros

dispositivos e atuadores, em que um sistema operacional completo seria algo exagerado para lidar com ações simples de leitura e resposta.

4 ALGUNS PRÉ-REQUISITOS

Com o crescimento exponencial de aplicações web, em específico no ecossistema da IoT, a comunidade começou com um movimento colaborativo para compartilhar o conhecimento e, nesse sentido, surgiram boas opções de aprendizagem sem ter real necessidade da aquisição do conjunto de hardware (Arduino + sensores + atuadores), e tem-se observado que não ter acesso ao hardware é a primeira barreira encontrada para aprender a programar.

Nada mais apropriado do que utilizar uma plataforma online que literalmente simula um circuito real e incorpora a programação também de acordo com o microcontrolador. A proposta, então, neste capítulo é apresentar a plataforma chamada **Tinkercad**, aplicada para o design de modelos 3D em CAD, assim como para a simulação de circuitos elétricos analógicos e digitais, desenvolvida pela Autodesk. E por ser gratuita e fácil de usar, é a opção recomendada para o aprendizado de Programação Embarcada através da linguagem C e por meio do Arduino.



Figura 5 – Plataforma online Tinker Cad
Fonte: Tinker Cad (2020)

5 EFETUANDO O CADASTRO NO TINKERCAD

Para acessar o Tinkercad, primeiramente você deve criar uma conta.

- Acesse o site www.tinkercad.com.
- Clique em “Inscrever-se” no canto superior direito da página inicial, conforme pode ser visto na figura abaixo.

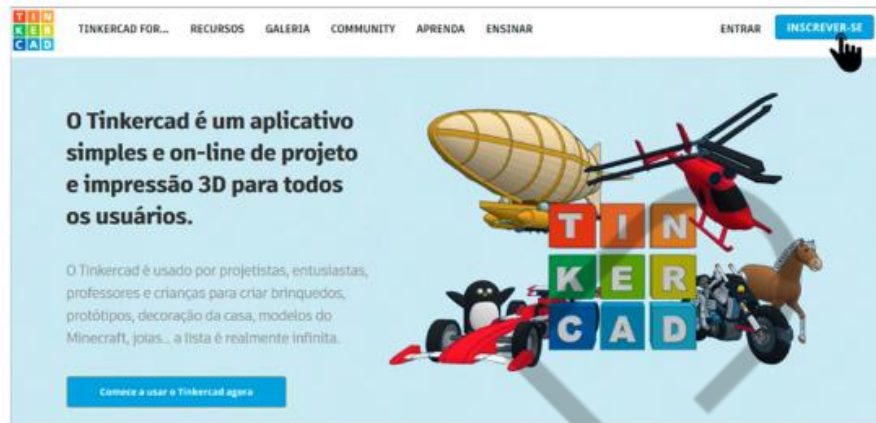


Figura 6 – Plataforma online Tinkercad
Fonte: Tinkercad (2020)

A partir deste ponto, é necessário seguir os passos representados na figura – “Passos para a criação de conta” – para criar sua conta, de modo simples e rápido:

Figura 7 – Passos para a criação de conta
Fonte: Tinkercad (2020)

Ao concluir o cadastro, você verá um tutorial de como funciona a ferramenta, facilitando ainda mais a criação de novos projetos, conforme observado na figura a seguir.

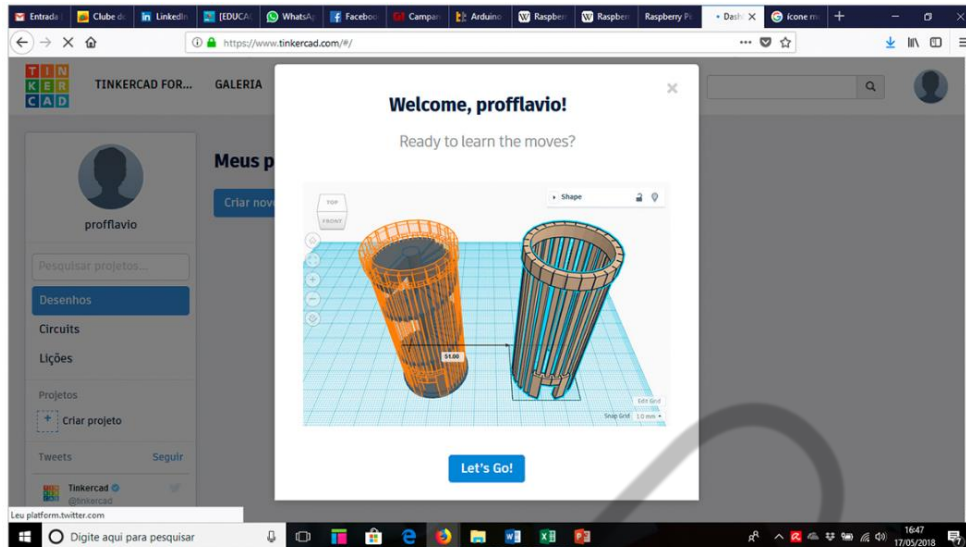


Figura 8 – Passos para a criação de conta
Fonte: Elaborada pelo autor (2020)

O tutorial está disponível na seção “Aprenda” do website, dividido em 2 categorias: 3D Design e Circuitos, com o passo a passo de como fazer os movimentos, projetos para treino e vídeos demonstrativos.

Após aprender como usar a ferramenta, está pronto para começar um projeto. Ao acessar a página Desenhos, você terá a lista dos seus projetos – desenhos e circuitos e o botão para criar um projeto ou, neste caso, um novo circuito, que pode ser visto na figura – “Criação de novos circuitos”.

Observe também, na figura abaixo, que se já existirem circuitos criados, eles aparecerão na tela principal, e somado a isso, é ainda possível compartilhar o circuito e torná-lo público, com o objetivo de aumentar a comunidade de usuários e facilitando a disseminação do conhecimento.

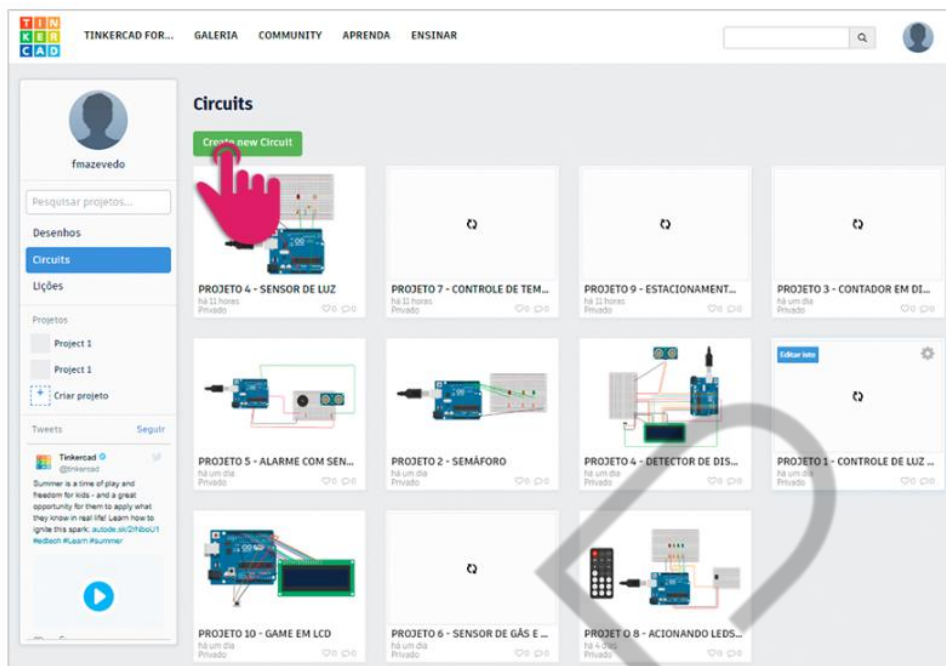


Figura 9 – Criação de novos circuitos
Fonte: Elaborado pelo autor (2020)

6 TREINAR É PRECISO

Embora a curiosidade para modelagem 3D seja grande, é fundamental que o foco esteja direcionado para o aprendizado de novos circuitos eletrônicos utilizando o Arduino. E sempre que é apresentada uma nova plataforma, tudo parece bastante complexo, por esse motivo, a recomendação antes de desenvolver os projetos é de utilizar a área “Aprenda”, mais especificamente os “circuits” (circuitos).

Na área “Aprenda” existe uma divisão que apresenta aspectos básicos (componentes eletrônicos e uso do simulador), descrição dos circuitos elétricos (noção de circuitos e Lei de Ohm) e os projetos desenvolvidos utilizando o Arduino.

A figura a seguir mostra o local exato em que as categorias mencionadas estão, e em cada uma delas as opções dos roteiros para que você conheça um pouco melhor da plataforma.

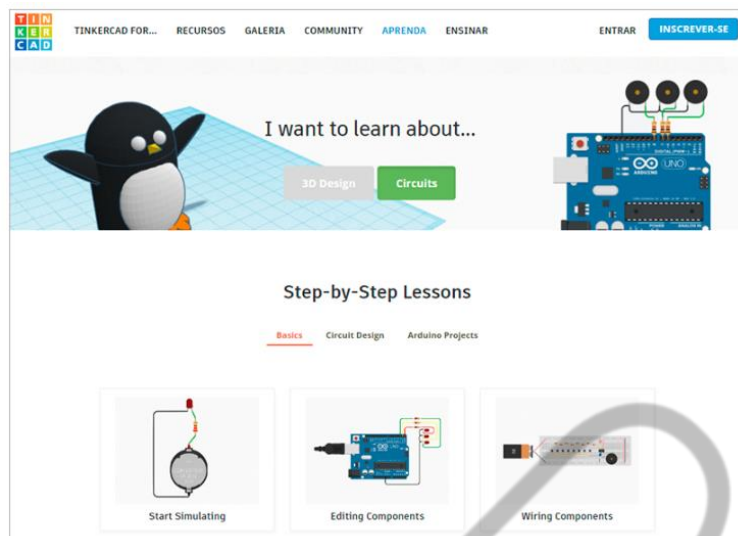


Figura 10 – Área Aprenda do Tinkercad
Fonte: Tinkercad (2020)

7 DESENVOLVENDO PROJETOS

7.1 Circuitos elétricos

A utilização do Arduino necessita de alguns conceitos prévios como tensão e corrente elétrica, já que “trabalha com lógica de 5 volts” e cada porta de saída pode fornecer até 40 mA de corrente elétrica.

A energia elétrica flui por intermédio do movimento dos elétrons livres, presentes nos materiais condutores elétricos, e provocada por uma elevação do potencial elétrico em um ponto desse condutor. Essa diferença no potencial elétrico de um ponto a outro chamamos de diferença de potencial ou tensão elétrica. Já a vazão de cargas elétricas, que se movimentam dentro do condutor em um certo ponto, chama-se de corrente elétrica.

Para que a energia elétrica seja utilizada é preciso criar uma diferença de potencial, que por sua vez imporá uma corrente elétrica a um material condutor. E a maneira como a energia elétrica é usada acontece por meio de um circuito elétrico, nele a corrente elétrica está sempre circulando devido a um fornecimento ininterrupto de tensão elétrica.

Comparação entre as transformações de energia em circuitos Hidráulico e Elétrico em paralelo

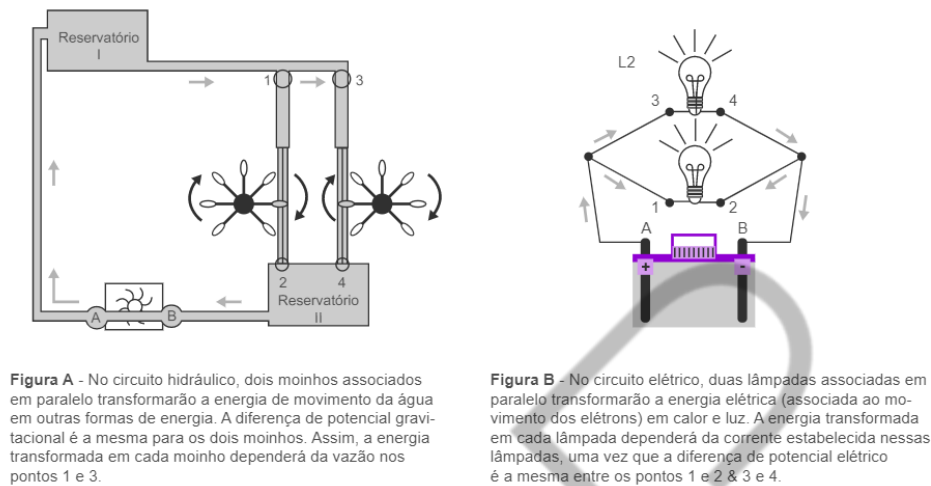


Figura 11 – Circuito Elétrico X Hidráulico
Fonte: Ponto Ciência (2020)

A figura anterior apresenta uma analogia bem interessante entre um circuito elétrico e um circuito hidráulico, e de acordo com ela, no circuito hidráulico, a energia hidráulica transformada em outras formas de energia pelos moinhos dependerá da diferença de potencial gravitacional entre os reservatórios. Já no circuito elétrico, a energia elétrica transformada em calor e luz por lâmpada dependerá da diferença de potencial elétrico entre os terminais de cada lâmpada.

8 PLACA PROTOBOARD, MATRIZ DE CONTATO OU BREADBOARD

Proteoboard consiste em uma placa didática composta de uma matriz de contatos, que permite a construção de circuitos sem a necessidade de efetuar a solda dos componentes. Tal fato permite que seja efetuada uma série de experimentos com os mesmos componentes, como se fosse um quebra-cabeças que pode ser montado e desmontado com rapidez e segurança.

Observe atentamente a figura abaixo e veja que cada fileira de 5 orifícios na coluna central forma uma trilha conectada e isolada das demais trilhas, e cada coluna de orifícios nas trilhas laterais forma uma trilha de alimentação, e está toda conectada. Geralmente, elas são ligadas à tensão de alimentação (5V) ou ao terra (GND ou 0V).

As seguintes “regras de ouro” são muito importantes quando o assunto é a montagem de circuitos em protoboard:

- Dois terminais de componentes plugados à mesma trilha estão eletricamente conectados.
- Nunca conectar dois terminais de um mesmo dispositivo na mesma trilha, pois assim eles estarão em curto-circuito.

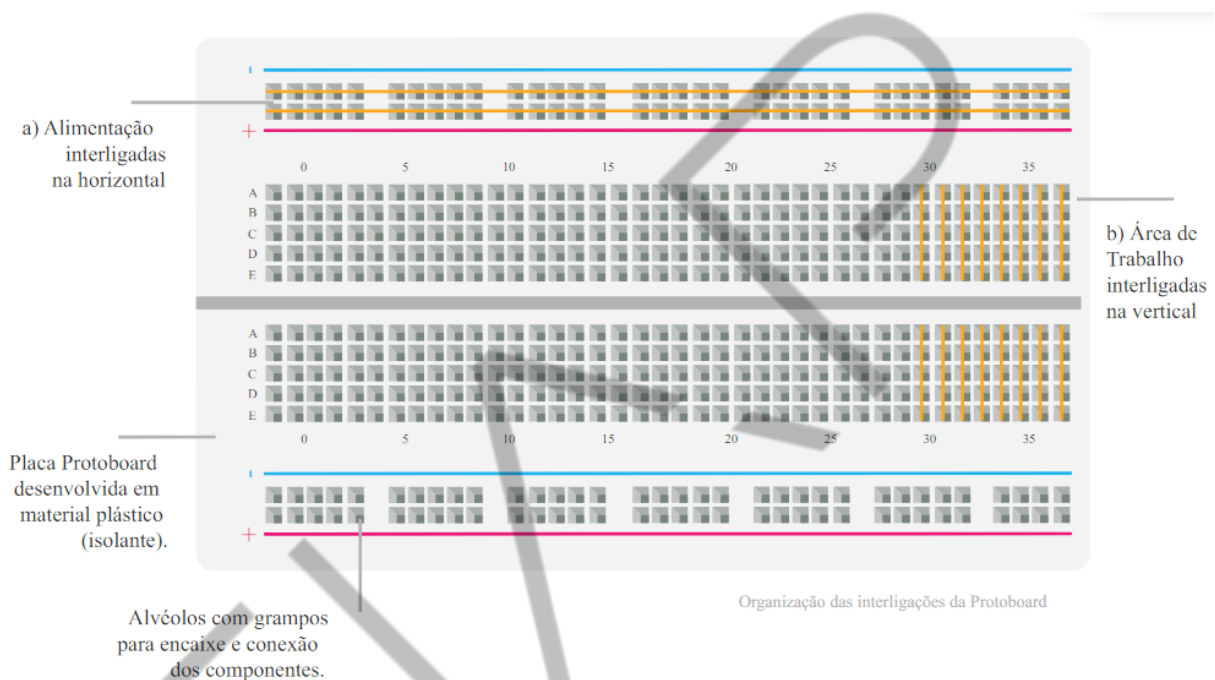


Figura 12 – Exemplo de Protoboard
Fonte: Dreaminc (2020)

9 OS SENSORES

Os sensores analógicos ou transdutores elétricos conseguem capturar um fenômeno físico e transformá-lo em um sinal elétrico (tensão ou corrente) que varia de forma análoga ao fenômeno.

Em geral, esses sensores não geram sua própria diferença de potencial, por isso dependem do fornecimento externo de tensão para gerar uma corrente, que varia de acordo com o fenômeno sendo medido.

Por exemplo, um microfone é um transdutor eletroacústico, pois transforma a variação da pressão atmosférica em uma corrente elétrica que varia da mesma forma.

E para capturar a medida do sensor, os circuitos digitais como o Arduino precisam medir o valor de tal corrente elétrica.

Os sensores digitais, por sua vez, podem assumir dois valores em seu sinal de saída ao longo do tempo, e que podem ser interpretados como 0 ou 1.

Tais valores não representam grandezas físicas, mas por meio de comparadores são identificados pelos sistemas de controle. Um bom exemplo para o uso de sensor digital é para a indicação de presença, como aqueles utilizados em alarmes ou acionamento de luz.

10 DESENVOLVIMENTO DE PROJETOS

A partir do conhecimento inicial da plataforma Tinkercad e criação de uma conta de usuário, é chegado o momento de desenvolver alguns projetos, que de forma linear são apresentados e descritos ao longo do texto, assim como o aumento do nível de complexidade computacional e de arquitetura de hardware (sensores e atuadores).

Nos próximos tópicos, serão apresentados respectivamente cada um dos projetos e seus respectivos códigos-fontes, seguidos de uma demonstração dos resultados dentro da plataforma.

10.1 Projeto 1 – Controle de Luz com Potenciômetro

Dando início ao processo de montagem do circuito e simulação, tem-se o projeto envolvendo o controle de luz (led) por meio de um potenciômetro (componente eletrônico que possui resistência elétrica ajustável por meio de um botão do tipo sintonizador).

Antes de descrever o circuito, neste primeiro projeto é fundamental apontar as regiões da plataforma responsáveis por algumas funcionalidades:

- **Circuit view** (visualização do circuito): região demarcada com a cor vermelha, esta funcionalidade retorna para o visual do circuito.
- **Components list** (lista dos componentes): região demarcada com a cor verde, quando acionada mostra a lista dos componentes utilizados no

projeto, algo bem útil e que orienta a compra dos elementos eletrônicos para um projeto no mundo real.

- **Code** (código): esta região, demarcada com a cor marrom, é fundamental para a interface entre o Arduino e os sensores e atuadores, e nela será inserido o código desenvolvido usando um conjunto de funções C / C ++ que podem ser utilizadas.
- **Start Simulation** (iniciar a simulação): região demarcada com a cor amarela, é responsável pela simulação em tempo real do projeto, e quando acionada, apresenta dinamicamente os estados dos sensores e atuadores, além de seus efeitos visuais.

Siga o modelo apresentado pela figura para que o circuito esteja em conformidade. É importante observar que ao lado direito da figura estão visíveis alguns componentes utilizados, como no caso o Arduino Uno, o potenciômetro, o led, e o resistor, além da protoboard, é claro.

Toda construção é baseada na escolha dos componentes por meio do *drag and drop* (ou clicar e arrastar) do mouse, bastando posicioná-los na protoboard ou se por um acaso errar a escolha, bastaria utilizar a tecla ESC para deixar de selecionar o respectivo componente.

O formato visual dos cabinhos coloridos (ou jumpers macho-macho) são obtidos dando-se um duplo clique no mouse para possibilitar os seus desvios da protoboard e Arduino.

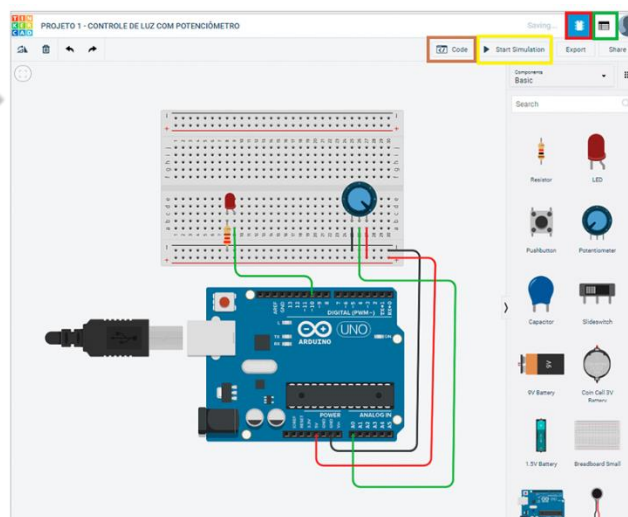


Figura 13 – Controle de luz com potenciômetro

Fonte: Elaborada pelo autor (2020)

Ainda usando a figura acima, é importante notar que o cabo de cor preta representa o “terra” (GND), o cabo de cor vermelha representa o VCC (5V), e o cabo de cor verde representa o valor analógico de 0 a 1023 que é enviado para o Arduino (entrada analógica A0).

O resistor em série com o led servirá para evitar a sua sobrecarga, evitando que ele queime; e o led, por sua vez, está conectado na porta digital ~D10. E o fato de ligar em uma porta digital com o símbolo ~ acontece devido as portas digitais (GPIO) marcadas com um ~ (til) possuírem a capacidade de servirem de saída do tipo PWM – Pulse-Width Modulation.

Uma saída PWM automaticamente alterna períodos em que ela está ligada (HIGH) e em que está desligada (LOW). A proporção do tempo em que a saída está ligada em relação ao tempo total do ciclo é chamado de *Duty Cycle*, e varia de 0 a 100%, ou no caso de 0 até 255.

Retomando a composição do circuito e aproveitando-se da funcionalidade “Components list”, a tabela abaixo reúne todos para facilitar a identificação e futura aquisição dos componentes.

Nome	Quantidade	Componente
Rpot1	1	10 kOhm, Potenciômetro
U1	1	Arduino Uno R3
D1	1	LED Vermelho
R2	1	Resistor de 220 ohm

Tabela 1 – Componentes do Projeto 1
Fonte: Elaborada pelo autor (2020)

O próximo passo, após o desenho do circuito, é o de desenvolver o código responsável pelo envio e recebimento dos dados pelo Arduino e a atuação de acordo com a necessidade do projeto.

Antes de implementar o código, é importante mostrar a estrutura mínima para o funcionamento, e no caso do Arduino são essenciais:

- **Função void setup():** chamada no início da execução do programa e deve configurar os dispositivos a serem usados.

- **Função void loop():** executada dentro do laço principal de execução e é chamada enquanto a placa estiver ligada.
- **Classe Serial:** representa a classe que se comunica com o computador hospedeiro através da porta USB/serial.
- **Função delay(milissegundos):** pausa a execução por X milissegundos.

Observe pelo código a seguir como está representada a estrutura básica para o funcionamento do Arduino:

```
void setup() {
  Serial.begin(9600);
}
void loop() {
  Serial.println("Hello World!");
  delay(1000);
}
```

Código-fonte 1 – Exemplo de código-fonte Arduino
Fonte: Elaborado pelo autor (2020)

Após as explicações sobre os elementos do circuito é chegado o momento de efetuar a simulação, e para tal é preciso antes clicar em “Code”, copiar e colar o seguinte código:

```
float entrada;
int led=10;
float saida;

void setup()
{
  Serial.begin(9600);
  pinMode(led,OUTPUT);
}

void loop()
{
  entrada=analogRead(A0);
  saida=map(entrada,0,1023,0,255);
  analogWrite(led,saida);
  Serial.print("Entrada : ");
  Serial.print(entrada );
  Serial.print("Saida : ");
  Serial.println(saida);
  delay(1000);
}
```

Código-fonte 2 – Controle de Luz com Potenciômetro
Fonte: Elaborado pelo autor (2020)

O código acima demonstra como declarar e inicializar variáveis, como é o caso da variável led que recebe o valor 10 (indicando a porta digital ~D10), e a variável entrada (porta analógica A0) que recebe os valores vindos do potenciômetro.



```

1 float entrada;
2 int led=10;
3 float saida;
4
5 void setup()
6 {
7   Serial.begin(9600);
8   pinMode(led,OUTPUT);
9
10 }
11
12 void loop()
13 {
14   entrada=analogRead(A0);
15   saida=map(entrada,0,1023,0,255);
16   analogWrite(led,saida);
17
18   Serial.print("Entrada : ");
19   Serial.print(entrada );
20   Serial.print("Saída : ");
21   Serial.println(saida);
22
23   delay(1000);
24 }
25
26
27
28

```

Serial Monitor

```

Entrada : 777.00Saída : 193.00
Entrada : 777.00Saída : 193.00
Entrada : 941.00Saída : 234.00
Entrada : 1023.00Saída : 255.00
Entrada : 1023.00Saída : 255.00
Entrada : 1023.00Saída : 255.00
Entrada : 798.00Saída : 198.00
Entrada : 573.00Saída : 142.00
Entrada : 573.00Saída : 142.00
Entrada : 368.00Saída : 91.00
Entrada : 368.00Saída : 91.00
Entrada : 368.00Saída : 91.00
Entrada : 368.00Saída : 91.00

```

Figura 14 – Simulação do Controle de luz com potenciômetro
Fonte: Elaborada pelo autor (2020)

A função analogWrite() envia para a porta ~D10 o valor mapeado de 0 até 1023 para 0 até 255. O final do código efetua a exibição dos valores na saída pela porta Serial com o comando Serial.print(saída), seguido de um intervalo de 1 segundo delay(1000), repetindo o processo enquanto o dispositivo estiver ligado.

Chegando quase ao final da execução do projeto, é preciso clicar em Start Simulation, e em Serial Monitor (na parte inferior logo abaixo do código) para visualizar

o resultado final, representado pela figura, por meio de um corte especial mostrando somente o código-fonte e a saída serial.

Com o processo de simulação acontecendo, você pode “brincar” e alterar os valores do potenciômetro, fazendo o movimento circular para aumentar e diminuir os valores, isso fará com que o resultado apresentado na saída serial seja consequentemente alterado.

10.2 Projeto 2 – controle de semáforo

Neste projeto será mostrado o princípio de funcionamento do controle semafórico, usando-se leds das cores amarela, verde e vermelha, e os resistores para que os respectivos leds associados não queimem.

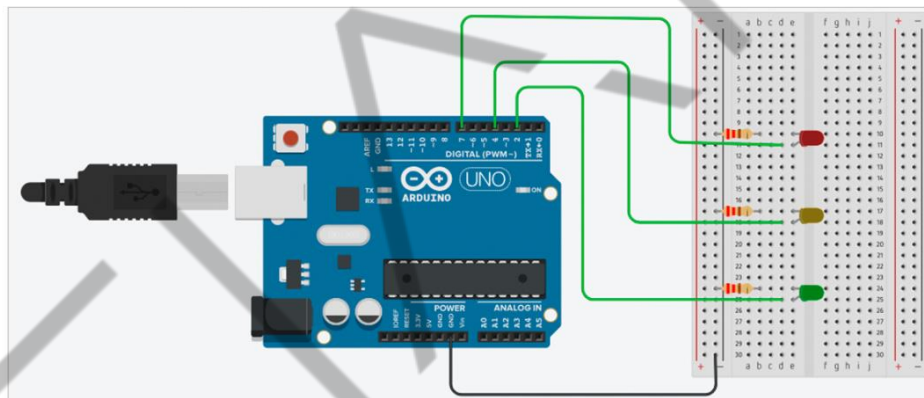


Figura 15 – Controle de Semáforo
Fonte: Elaborada pelo autor (2020)

O código utilizado com a exemplificação de um semáforo pode ser obtido pelo código-fonte a seguir:

```
/** Definições */
int vermelho=7; //definição da porta para o led vermelho
int amarelo=4; //definição da porta para o led amarelo
int verde=2; //definição da porta para o led verde
/** Programa */
void setup() {
  pinMode(verde,OUTPUT); //direcionar a porta verde
  pinMode(amarelo,OUTPUT); //direcionar a porta amarela
  pinMode(vermelho,OUTPUT); //direcionar a porta vermelha
}

void loop() {
  digitalWrite(verde,HIGH); //acender o led verde
```

```

delay(5000);           //esperar 5 segundos
digitalWrite(verde,LOW); //apagar o led verde
delay(500);
digitalWrite(verde,HIGH); //acender o led verde
delay(500);           //esperar meio segundo
digitalWrite(verde,LOW); //apagar o led verde
delay(500);           //aguardar meio segundo
digitalWrite(verde,HIGH); //acender o led verde
delay(500);           //esperar meio segundo
digitalWrite(verde,LOW); //apagar o led verde
delay(500);           //esperar meio segundo

digitalWrite(amarelo,HIGH); //acender o led amarelo
delay(500);           //esperar meio segundo
digitalWrite(amarelo,LOW); //apagar o led amarelo
delay(500);           //aguardar meio segundo
digitalWrite(amarelo,HIGH); //acender o led amarelo
delay(500);           //aguardar meio segundo
digitalWrite(amarelo,LOW); //apagar o led amarelo
delay(500);           //aguardar meio segundo
digitalWrite(amarelo,HIGH); //acender o led amarelo
delay(500);
digitalWrite(amarelo,LOW); //apagar o led amarelo
delay(500);           //aguardar meio segundo

digitalWrite(vermelho,HIGH); //acender o led vermelho
delay(5000);           //esperar 5 segundos
digitalWrite(vermelho,LOW); //apagar o led vermelho
delay(500);           //aguardar meio segundo
}

```

Código-fonte 3 – Controle de semáforo
 Fonte: Elaborado pelo autor (2020)

O princípio de funcionamento do programa leva em consideração o acendimento do led por meio da função digitalWrite() nas portas digitais 2, 4 e 7, cada uma delas representando uma cor.

E o acendimento do led é seguido de uma função que o mantém acesso por alguns segundos, de acordo com a necessidade de espera do motorista para seguir viagem ou aguardar a sua vez.

É importante lembrar que para visualizar alguma saída é preciso gerar a impressão para a interface serial, caso contrário, é difícil identificar algum problema quanto aos valores recebidos através dos sensores.

Novamente, no sentido de oferecer uma lista com os elementos utilizados, e também pelo fato do circuito ser bastante simples, utilizando a funcionalidade “Components List”, é possível gerar a tabela a seguir:

Nome	Quantidade	Componente
U1	1	Arduino Uno R3
D1	1	LED Vermelho
D2	1	LED Amarelo
D3	1	LED Verde
R1	3	Resistor de 220 ohm
R2		
R3		

Tabela 2 – Componentes do Projeto 2
Fonte: Elaborada pelo autor (2020)

10.3 Projeto 3 – Alarme com sensor ultrassônico

Este próximo projeto leva em consideração um sensor de distância usando ultrassom, e que de acordo com a distância do objeto, dispara um sinal sonoro enquanto o objeto estiver em um perímetro de alcance.

De um modo geral é bastante simples, e será dado ênfase a não somente encaminhar a programação usando texto assim como usar o conceito de programação em blocos, típico do aplicativo Scratch do MIT.

Observe a figura a seguir, que mostra o circuito utilizado para a execução do projeto.

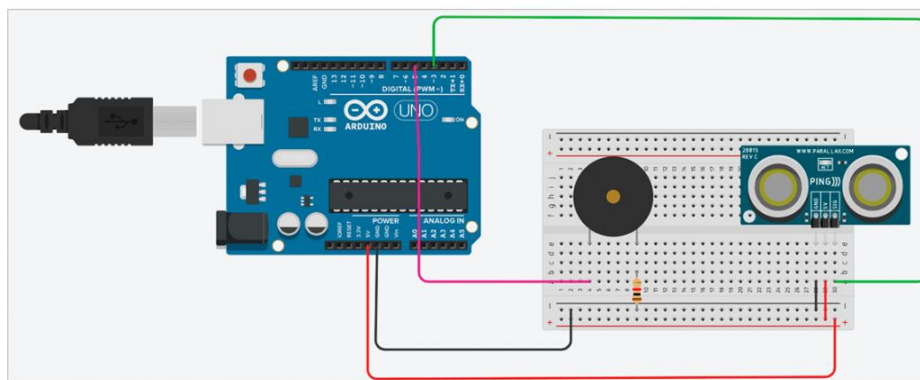


Figura 16 – Alarme com sensor ultrassônico
Fonte: Elaborada pelo autor (2020)

Quanto ao desenvolvimento do código, a plataforma não somente permite a inserção usando texto, pois além do texto, é possível usar a notação de programação em blocos, facilitando o desenvolvimento para aqueles que ainda estão começando com a parte de programação, portanto, vale a pena apresentar esta funcionalidade.

Clicando-se na funcionalidade Code, são permitidas 3 opções, usar código ou blocos ou simultaneamente os dois, porém não é permitido alterar de um para o outro sem que ocorra a perda do que já existe, portanto é fundamental seguir uma linha no desenvolvimento.

Caso seja escolhida a opção Blocks, a seguinte estrutura será oferecida para você, e que pode ser explorada de acordo com os itens: *Output*, *Input*, *Control*, *Math*, *Notation* e *Variables*. Elementos estes que estão presentes quando o código é criado manualmente.

A utilização da programação em blocos já faz a geração do código para ser executado no Arduino, portanto, não é possível fazer alterações nele, e isso faz com que todos os comentários e estrutura estejam sempre na língua inglesa, o que, por sinal, não é um impedimento para a aprendizagem.

Com o intuito de mostrar o modo de programação por blocos, a figura abaixo mostra como seria a organização visual e sequencial para a execução do projeto.

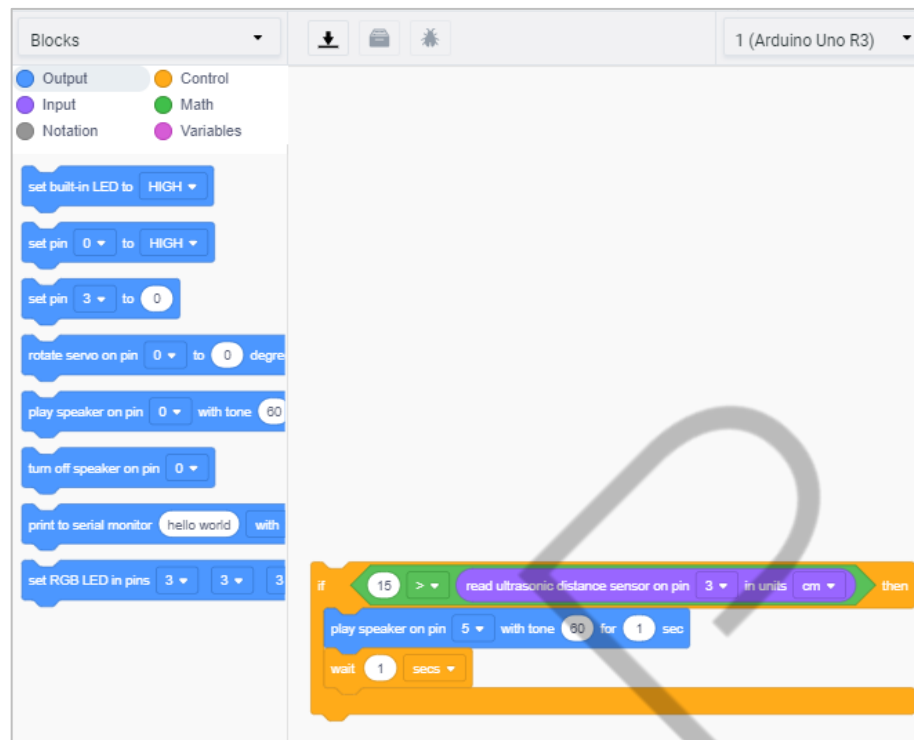


Figura 17 – Programação por blocos

Fonte: Elaborada pelo autor (2020)

Em se tratando do código-fonte a seguir, gerado pela programação por blocos, é possível notar que o acionamento do “*buzzer*” (elemento sonoro) acontece quando a distância de algum objeto é menor do que 15 centímetros e dura enquanto o objeto estiver nesse perímetro.

O projeto se assemelha a um sistema de detecção de obstáculos inserido nas extremidades dos carros para indicar ao motorista que algo se aproxima quando a manobra acontece.

```
long readUltrasonicDistance(int pin)
{
  pinMode(pin, OUTPUT); // Clear the trigger
  digitalWrite(pin, LOW);
  delayMicroseconds(2);
  // Sets the pin on HIGH state for 10 micro seconds
  digitalWrite(pin, HIGH);
  delayMicroseconds(10);
  digitalWrite(pin, LOW);
  pinMode(pin, INPUT);
  // Reads the pin, and returns the sound wave travel time in
  microseconds
  return pulseIn(pin, HIGH);
}

void setup()
```

```
{
  pinMode(3, INPUT);
  pinMode(5, OUTPUT);
}

void loop()
{
  if (15 > 0.01723 * readUltrasonicDistance(3)) {
    tone(5, 523, 1000); // play tone 60 (C5 = 523 Hz)
    delay(1000); // Wait for 1000 millisecond(s)
  }
}
```

Código-fonte 4 – Alarme com sensor ultrassônico
Fonte: Elaborada pelo autor (2020)

E quanto ao resultado visual? Como fazer para ter a percepção de que um objeto se aproxima? Neste caso, é interessante mostrar que ao clicar no componente PING1 (sensor ultrassônico) é possível aproximar e distanciar um objeto para que a condição prevista no código aconteça, proporcionando testes extremamente próximos ao mundo real.

Observe na figura a seguir o resultado visual e sensorial em um determinado momento, em que a condição de acionamento do alarme acontece.

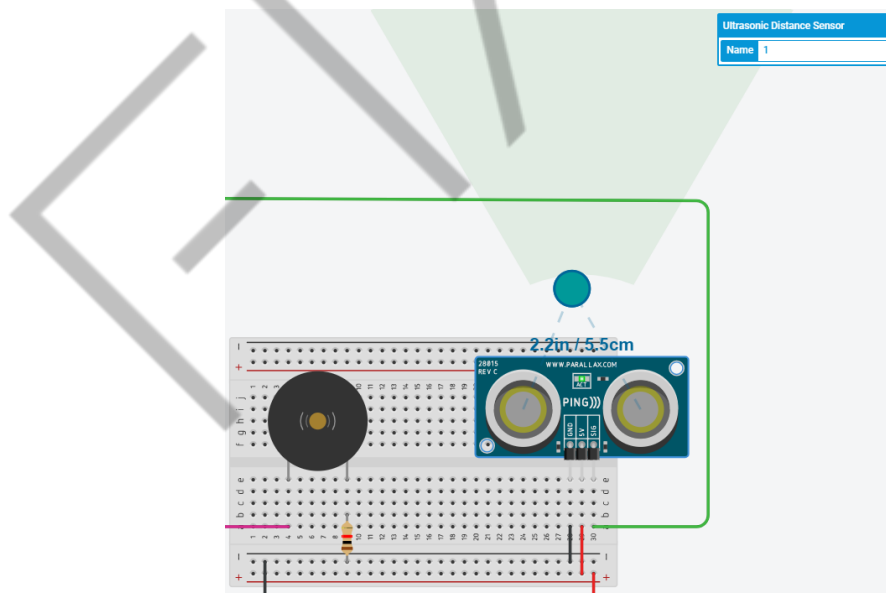


Figura 18 – Simulação do alarme com sensor ultrassônico
Fonte: Elaborada pelo autor (2020)

E para finalizar a apresentação do projeto, segue a lista com os componentes utilizados para realizá-lo, a partir da tabela:

Nome	Quantidade	Componente
U1	1	Arduino Uno R3
PING1	1	Sensor de Distância Ultrassônico
PIEZO1	1	Piezo (buzzer)
R1	1	Resistor de 1 kohm

Tabela 3 – Componentes do Projeto 3
Fonte: Elaborada pelo autor (2020)

11 CONSTRUÇÃO DO GATEWAY COM O MQTT

Antes de implementar a solução que enviará os dados para uma plataforma na nuvem, é preciso apresentar alguns protocolos responsáveis pelas trocas de mensagens entre dispositivos **IoT** e a Internet.

Em particular, são detalhados dois protocolos de mensagens que atendem as necessidades dos dispositivos **IoT**. O primeiro deles, denominado *Message Queuing Telemetry Transport* (MQTT), é antigo para os padrões de hoje, e teve sua origem em 1999. E o segundo, o de aplicação restrita (CoAP ou *Constrained Application Protocol*), é relativamente novo e está ganhando notoriedade atualmente.

E se de um lado existem os protocolos, de outro deve haver um conjunto de regras que norteiam o formato das mensagens trocadas pelos respectivos protocolos, e nesse sentido é importante introduzir o papel dos servidores de rede.

Os servidores da rede podem se comunicar com os dispositivos ou aplicações conectadas por diversos protocolos de mensagens que funcionam bem para comandos simples. E dentro do conceito de simplicidade, são necessários protocolos novos, mais leves e que não exigem grande quantidade de recursos, como é o caso dos protocolos MQTT e CoAP, que atendem a essas necessidades por meio de tamanhos reduzidos no formato e gerenciamento das mensagens, além da menor sobrecarga.

Em termos de formato, as mensagens trocadas nesse contexto, em geral, são estruturadas de acordo com regras que permitem o uso de interpretadores padrão:

- **XML**: campos do documento definidos através de markups.

- **HTML:** subconjunto do XML usado para páginas web.
- **JSON:** documento definido como um objeto JavaScript.

11.1 Sobre o MQTT

A IBM criou o protocolo MQTT (*Message Queue Telemetry Transport* ou Fila de Mensagem para Transporte de Telemetria) para comunicações via satélite com equipamentos de campo de petróleo. O MQTT tinha confiabilidade e baixa potência em seu núcleo e, portanto, fazia todo o sentido ser aplicado às redes de **IoT**.

O protocolo MQTT é muito simples para publicação e recebimento de mensagens, e bastante apropriado para dispositivos com alta latência e baixa largura de banda de comunicação, o que significa que "algo" publica mensagens em algum lugar e outro "algo" subscreve, recebendo as mensagens publicadas.

Nesse tipo de protocolo não há comunicação direta entre o emissor e o receptor da mensagem, ou seja, não é ponto a ponto. A leveza do protocolo, que usa cabeçalhos de poucos bytes, torna-o adequado para a comunicação de objetos no cenário da internet das coisas.

O processo de comunicação é feito com um servidor, representado por um aplicativo que recebe as mensagens publicadas e as entrega aos subscritores correspondentes. Tal servidor é chamado de broker (intermediário ou corretor, em português).

Os clientes MQTT são as pontas da comunicação, podendo enviar ou receber mensagens através das operações:

- **Publish:** um cliente MQTT publica uma mensagem com determinado tópico.
- **Subscribe:** um cliente se cadastra no servidor para receber cópias de mensagens com determinado tópico.

Em linhas gerais, o protocolo MQTT apresenta uma série de pontos fortes, dentre eles:

- **Modelo publicar / assinar modelo:** o modelo "pub / sub" do MQTT tem bom escalonamento e pode ser eficiente em termos de energia. Os nós da rede publicam dados e outros assinam de acordo com o conteúdo, tipo ou

assunto da mensagem. Geralmente, o *broker* assina todas as mensagens e, em seguida, gerencia o fluxo de informações para seus nós.

- **Desacoplamento do espaço:** embora o nó e o intermediário precisem ter o endereço IP um do outro, os nós podem publicar informações e assinar as informações publicadas de outros nós sem nenhum conhecimento um do outro, já que tudo passa pelo intermediário central. Isso reduz a sobrecarga que pode acompanhar as sessões e portas TCP e permite que os nós finais operem independentemente uns dos outros.
- **Desacoplamento do tempo:** um nó pode publicar suas informações, independentemente dos estados de outros nós. Outros nós podem, então, receber os dados publicados do intermediário quando eles estiverem ativos. Isso permite que os nós permaneçam em stand by, mesmo quando outros nós estiverem publicando mensagens diretamente relevantes para eles.
- **Desacoplamento de sincronização:** um nó que está no meio de uma operação não é interrompido para receber uma mensagem publicada à qual está inscrito. A mensagem é enfileirada pelo intermediário até que o nó receptor seja concluído com sua operação existente. Isso economiza corrente de operação e reduz operações repetidas, evitando interrupções de operações em andamento ou “estados sonolentos”.
- **Segurança:** o MQTT usa TCP não criptografado e não é seguro para uso imediato, no entanto, como usa o TCP, ele pode e deve usar a segurança de Internet TLS / SSL. O TLS é um método muito seguro para criptografar o tráfego, mas também é intensivo em recursos para clientes leves devido ao handshake necessário e ao aumento da sobrecarga de pacotes. Para redes onde a energia é uma prioridade muito alta e a segurança é muito menor, criptografar apenas a carga útil do pacote pode ser suficiente.
- **Níveis de Qualidade de Serviço (QoS) MQTT:** o termo “QoS” significa outras coisas fora do MQTT. No MQTT, os níveis 0, 1 e 2 de “QoS” descrevem níveis crescentes de entrega garantida de mensagens.
- **MQTT QoS Nível 0 (no máximo uma vez):** comumente conhecido como “Disparar e esquecer”, trata-se de um único estouro de transmissão sem

garantia de chegada de mensagem. Pode ser usado para mensagens altamente repetitivas ou mensagens que não são de missão crítica.

- **MQTT QoS Nível 1 (pelo menos uma vez):** este nível tenta garantir que uma mensagem seja recebida pelo menos uma vez pelo destinatário pretendido. Quando uma mensagem publicada é recebida e entendida pelo destinatário pretendido, ela reconhece a mensagem com uma mensagem de confirmação (PUBACK) endereçada ao nó de publicação. Até o PUBACK ser recebido pelo publicador, ele armazena a mensagem e a retransmite periodicamente. Esse tipo de mensagem pode ser útil para um desligamento de nó não crítico.
- **MQTT QoS Nível 2 (exatamente uma vez):** este nível tenta garantir que a mensagem seja recebida e decodificada pelo destinatário pretendido. É o nível de QoS mais seguro e confiável do MQTT. O editor envia uma mensagem anunciando que tem uma mensagem de nível 2 de QoS. O destinatário pretendido reúne o anúncio, decodifica-o e indica que está pronto para recebê-la. O editor retransmite sua mensagem. Depois que o destinatário a entende, ela conclui a transação com uma confirmação. Esse tipo de mensagem pode ser útil para ligar ou desligar luzes ou alarmes em uma casa.

Do mesmo modo que os pontos fortes são citados, é imprescindível saber que existem gargalos ou pontos fracos no que se refere ao uso do protocolo MQTT, e dentre eles tem-se:

Broker central: o uso de um intermediário central pode ser uma desvantagem para sistemas **IoT** distribuídos. Por exemplo, um sistema pode começar pequeno com um controle remoto e o fechamento de uma janela com persianas, não necessitando, portanto, de um intermediário central. Então, à medida que o sistema cresce, por exemplo, adicionando sensores de segurança, lâmpadas ou outras persianas, a rede naturalmente cresce e se expande, e pode ter necessidade de um intermediário central. No entanto, nenhum dos nós individuais deseja assumir o custo e a responsabilidade, pois exige recursos, software e complexidade que não são essenciais para a função do nó final.

- **TCP:** o TCP foi originalmente projetado para dispositivos com mais recursos de memória e processamento que podem estar disponíveis em uma rede leve, como o caso da **IoT**. Por exemplo, o protocolo TCP exige que as conexões sejam estabelecidas em um processo de *handshake* de várias etapas antes que as mensagens sejam trocadas. Isso aumenta o tempo de ativação e de comunicação e reduz a vida útil da bateria em longo prazo. Além disso, no TCP, é ideal para dois nós de comunicação manterem seus soquetes TCP abertos um para o outro continuamente com uma sessão persistente, o que pode ser difícil com dispositivos com restrição de energia e recursos.

11.2 Sobre o CoAP

Conforme definido pelo IETF, o CoAP é para “uso com nós restritos e redes restritas (por exemplo, baixa potência, com perdas)”. A comunidade **Eclipse** também suporta CoAP como um padrão aberto e, como o MQTT, o CoAP é comercialmente suportado e cresce rapidamente entre provedores de **IoT**.

O CoAP é um protocolo cliente/servidor e fornece um modelo de interação “solicitação/relatório” individual, e ao contrário do MQTT, que foi adaptado às necessidades de **IoT**, o IETF especificou o CoAP desde o início para oferecer suporte a **IoT** com mensagens leves para dispositivos restritos operando em um ambiente restrito. O CoAP é projetado para interoperar com o HTTP e a Web RESTful por meio de proxies simples, tornando-o compatível com a Internet.

E novamente, assim que um protocolo é apresentado, é importante citar os pontos fortes e fracos.

Pontos fortes:

- **UDP nativo:** o CoAP é executado sobre o UDP (*User Datagram Protocol* ou Protocolo de Datagrama de Usuário), que é inerentemente e intencionalmente menos confiável do que o TCP, dependendo das mensagens repetitivas para confiabilidade em vez de conexões consistentes. Por exemplo, um sensor de temperatura pode enviar uma

atualização a cada poucos segundos, mesmo que nada tenha mudado de uma transmissão para a próxima. Se um nó receptor perder uma atualização, a próxima chegará em poucos segundos e, provavelmente, não será muito diferente da primeira.

- **Suporte multicast:** uma rede CoAP é inerentemente um para um, no entanto, ele suporta requisitos multicast de um para muitos ou de muitos para muitos. Isso é inerente ao CoAP porque é construído sobre o IPv6, que permite o endereçamento multicast para dispositivos além dos endereços IPv6 normais.
- **Segurança:** CoAP usa o DTLS sobre o protocolo de transporte UDP. Como o TCP, o UDP não é criptografado, mas pode e deve ser aumentado com o DTLS.
- **Descoberta de recursos/serviços:** o CoAP usa URI para fornecer uma apresentação padrão e expectativas de interação para os nós da rede. Isso permite um grau de autonomia nos pacotes de mensagens, uma vez que os recursos do nó de destino são parcialmente compreendidos por seus detalhes de URI. Em outras palavras, um nó sensor alimentado por bateria pode ter um tipo de URI, enquanto um atuador de controle de fluxo alimentado por linha pode ter outro. Os nós que se comunicam com o nó do sensor alimentado por bateria podem ser programados para esperar tempos de resposta mais longos, informações mais repetitivas e tipos de mensagem limitados. Os nós que se comunicam com o atuador de controle de fluxo acionado por linha podem ser programados para esperar mensagens ricas e detalhadas em um ritmo muito rápido.
- **Comunicação assíncrona:** dentro do protocolo CoAP, a maioria das mensagens é enviada e recebida usando o modelo request/report; no entanto, outros modos de operação permitem que os nós sejam um pouco desacoplados. Por exemplo, o CoAP tem um mecanismo de “observação” simplificado semelhante ao pub/sub do MQTT, que permite que os nós observem outros sem ativá-los.

Como não existem somente pontos fortes em todo e qualquer protocolo, é chegado o momento de mostrar os pontos fracos do protocolo CoAP.

Pontos fracos:

- **Maturidade padrão:** o MQTT é atualmente um padrão mais maduro e estável do que o CoAP. Para muitos desenvolvedores de **IoT**, é mais fácil colocar uma rede MQTT em funcionamento muito rapidamente do que uma rede semelhante usando o CoAP. Por esse motivo, o CoAP tem um tremendo momento de mercado e está evoluindo rapidamente para fornecer uma base padronizada. É provável que a CoAP atinja um nível de estabilidade e maturidade similar ao MQTT no curto prazo.
- **Confiabilidade e previsibilidade da mensagem (nível de QoS):** a “confiabilidade” do CoAP é a QoS do MQTT. Ele fornece um método muito simples de fornecer uma mensagem “confirmable” e uma mensagem “non-confirmable”. A primeira é confirmada com uma mensagem (ACK) do destinatário pretendido. Ou seja, confirma que a mensagem foi recebida, mas não confirma que o seu conteúdo foi decodificado corretamente ou de forma alguma. Uma mensagem não confirmada é “disparar e esquecer”.

11.3 Construção do gateway

Conforme citado anteriormente, um gateway **IoT** é um dispositivo intermediário entre sensores e dispositivos e os aplicativos que criam valor a partir de seus dados e acesso. O gateway permite coletar e transportar dados com segurança dos dispositivos, usuários remotos e aplicativos para atender a uma necessidade específica.

Como é bastante conhecido, o ecossistema **IoT** é composto por muitos componentes ou camadas. A camada inferior contém os sensores e dispositivos usados para medir o mundo e efetuar mudanças. O gateway atua como um intermediário seguro entre esses sensores e dispositivos e a nuvem (por meio de redes e, eventualmente, persistência em um sistema de armazenamento). A camada superior lida com o monitoramento e o gerenciamento geral do ecossistema da **IoT**, com análises dos dados coletados e persistentes, incluindo aplicativos e o valor que o ecossistema da **IoT** expõe.

Observe a figura a seguir, a qual ilustra o posicionamento do que é conhecido como Gateway **IoT**:

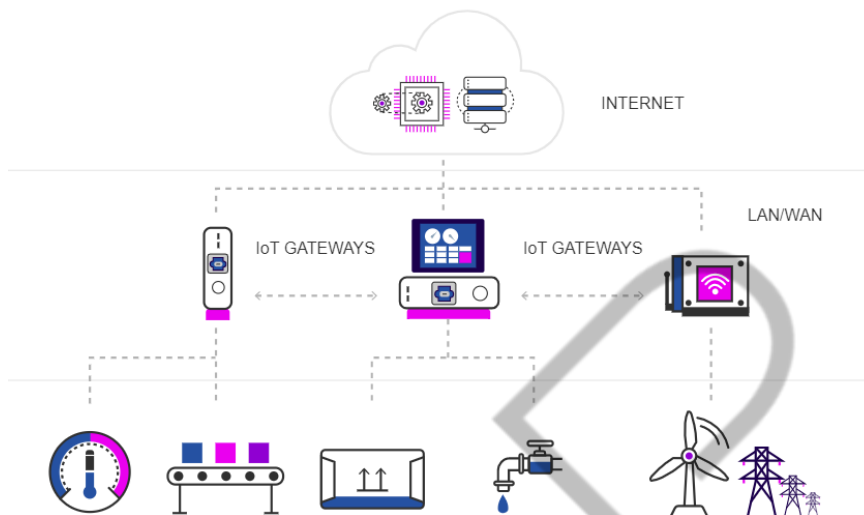


Figura 19 – Inclusão do Gateway IoT
Fonte: IOT Learners (2020)

A partir deste momento, para a construção de um gateway será necessário não somente conhecer a parte de programação para o Arduino, é também importante conhecer um pouco da linguagem de programação *JavaScript*. A importância é devido ao fato de que o envio dos dados será feito usando a interface serial do computador em que o Arduino está acoplado para um servidor Node.js, isto é, um ambiente de execução em tempo real para a linguagem *JavaScript*, com foco na execução de programas no lado do servidor.

A sequência didática seguirá um processo de recapitular os elementos do Arduino, mas desta vez não através da plataforma Tinkercad, e sim através de uma IDE (*Integrated Development Environment* ou Ambiente de Desenvolvimento Integrado) chamada Arduino IDE, que pode ser obtida em [ARDUNIO IDE](#).

Após apresentar um pouco sobre o ambiente de desenvolvimento, será preciso conhecer alguns conceitos sobre a linguagem JavaScript e como habilitar um servidor Node.js para que o Node-Red seja utilizado como parte da estruturação do gateway.

É importante lembrar também o papel do Gateway na **IoT**: prover, de modo simples e eficiente, com alta disponibilidade, serviços demandando pouco esforço computacional.

11.4 Arduino IDE

O Arduino IDE é uma aplicação multiplataforma escrita em Java derivada dos projetos Processing e Wiring, e inclui um editor de código com recursos de realce de sintaxe capaz de compilar e carregar programas para a placa de forma rápida e transparente.

Assim que o editor Arduino IDE estiver instalado, é importante ter um primeiro contato com a interface, pois mesmo simples, as suas funcionalidades devem ser bem conhecidas. Ao iniciar o aplicativo, a tela inicial que será visualizada pode ser comparada com a figura a seguir.

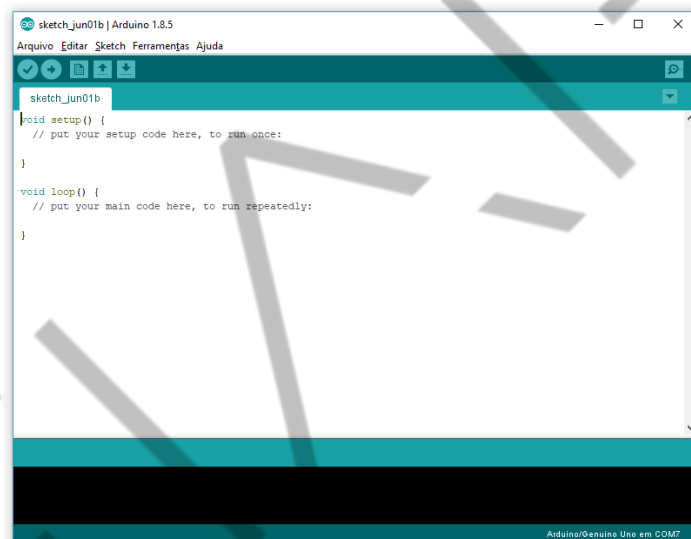


Figura 20 – Tela inicial do Arduino IDE
Fonte: Elaborada pelo autor (2020)

De um modo abrangente, as funcionalidades presentes no Arduino IDE são:

- **Arquivo:** item que reúne as possibilidades de criar, salvar, abrir exemplos existentes, definir algumas preferências do editor e imprimir o código atual.
- **Editar:** neste item, é possível atuar diretamente na edição do código, aumentando ou diminuindo a fonte, localizando caracteres e melhorando a exibição do código por meio de indentação.
- **Sketch:** é uma espécie de parte central da IDE, pois neste item é possível compilar, incluir bibliotecas e carregar o código diretamente no Arduino.
- **Ferramentas:** neste item é possível escolher a placa pelo gerenciador de placas, usar o monitor serial para verificar os dados enviados pela serial,

fazer a atualização de *firmware* da placa e identificar qual porta serial está sendo utilizada pela placa.

- **Ajuda:** trata-se de um tutorial elaborado em html com uma vasta explicação detalhada sobre a IDE, somada aos conceitos de programação para o Arduino.

11.5 Compilando o primeiro programa

Para que seja possível executar todos os procedimentos contidos no texto, é preciso ter “em mãos” alguns dispositivos e componentes eletrônicos, dentre eles um Arduino UNO, resistores e sensores.

O primeiro exemplo, somente para ilustrar, está relacionado com o acendimento de um led, e pela figura a seguir tem-se uma exemplificação de como ele precisa ser montado.

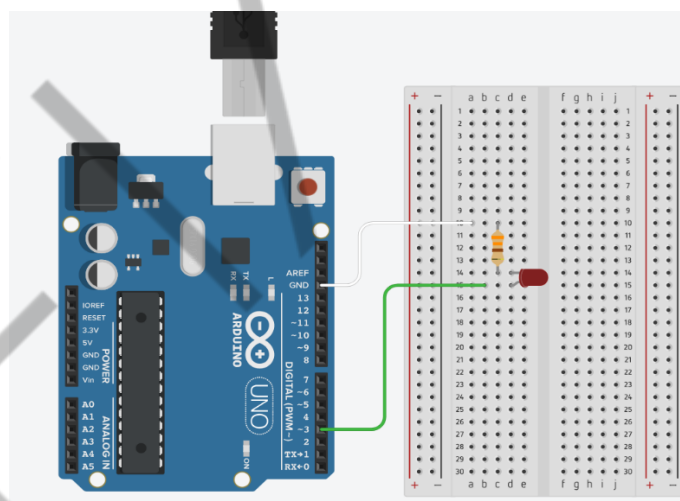


Figura 21 – Diagrama para o pisca led

Fonte: Elaborada pelo autor (2020)

Para que o circuito funcione, é necessário ligar o Arduino na interface USB do microcomputador, e já ter a montagem em que o led fica inserido em série com o resistor (330 ohms).

DICA: O papel do resistor é evitar a sobrecarga de tensão no led, evitando a sua queima. E em relação ao led, o seu polo positivo ou Anodo, deve ser ligado à porta controladora, enquanto o Catodo ou polo negativo, deve ser ligado ao GND (*ground* ou terra).

Observe pela figura a seguir, a montagem final do circuito, faltando apenas o código que enviará os comandos do Arduino para o acionamento do led.

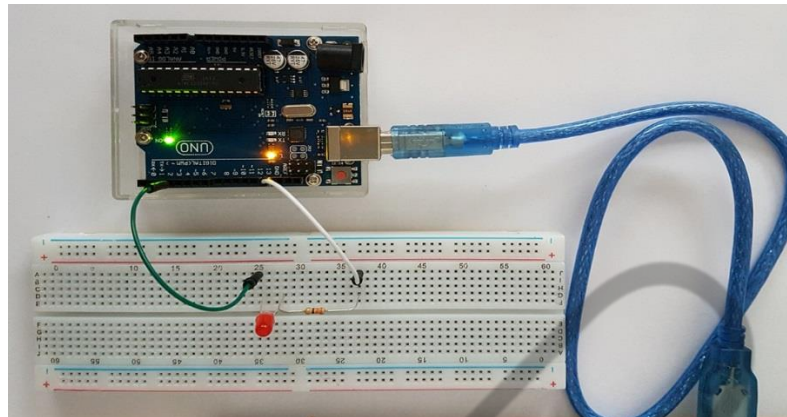


Figura 22 – Circuito real para o pisca led
Fonte: Elaborada pelo autor (2020)

Para finalizar e fazer efetivamente o led piscar, é preciso ter o código escrito em C para ser enviado para o Arduino, e nada mais adequado neste momento do que utilizar o próprio editor do Arduino IDE, conforme pode ser visto adiante.

```

pisca_led | Arduino 1.8.5
Arquivo Editar Sketch Ferramentas Ajuda

pisca_led

int led = 3; //Pino em que o led está conectado

void setup() {
  Serial.begin(9600);
  pinMode(led, OUTPUT);
}

void loop() {
  analogWrite(led, 255);
  delay(1000);
  analogWrite(led, 0);
  delay(1000);
}

Carregado
O sketch usa 1952 bytes (6%) de espaço de armazenamento para programas. O máximo são 32
Variáveis globais usam 184 bytes (8%) de memória dinâmica, deixando 1864 bytes para var
9
Arduino/Genuino Uno em COM7
  
```

Figura 23 – Código para o pisca led
Fonte: Elaborada pelo autor (2020)

Antes de enviar o código, é preciso, dentro da funcionalidade “Ferramentas”, conforme a exemplificamos na figura a seguir, selecionar a placa e a porta corretamente, pois caso isso não seja feito haverá um erro e o código não será enviado para o Arduino.

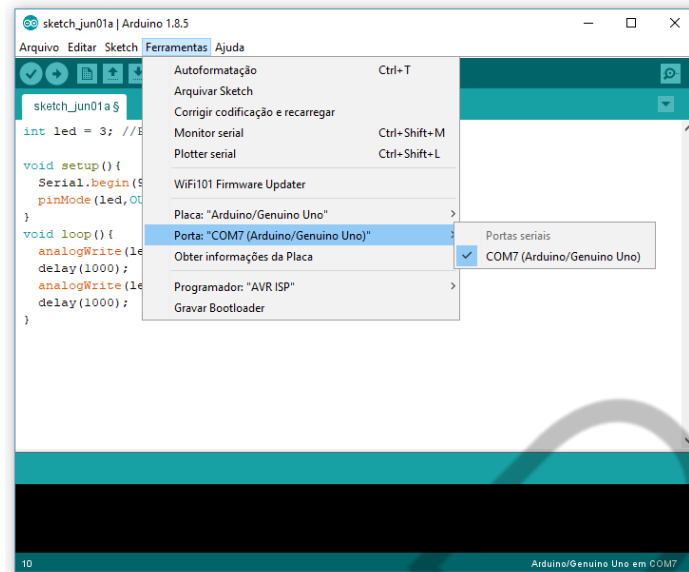


Figura 24 – Escolha da placa e porta
Fonte: Elaborada pelo autor (2020)

Após ter escolhido a placa e a porta corretamente, é preciso carregar o código, por meio da funcionalidade Sketch, e opção carregar. Quando a mensagem indicar “carregado”, isso significa que o código foi gravado no Arduino e ele passa a cumprir o que estiver determinado; e no caso, o led vermelho acenderá por 1 segundo e apagará por 1 segundo, dando o efeito de piscar enquanto o Arduino estiver ligado e enquanto um novo código não for inserido na placa.

11.5.1 Inserindo uma nova biblioteca

De acordo com o sensor que precisa ser utilizado no projeto, é preciso incluir uma biblioteca para seu pleno funcionamento. E como fator motivador para entender tal necessidade, será introduzido um novo projeto que utilizará o sensor de umidade e temperatura DHT, cuja biblioteca pode ser obtida em: <http://hobbyist.co.nz/sites/default/files/WeatherStation/DHT.zip>.

Após descompactar o arquivo na pasta “libraries”, que por sua vez encontra-se na pasta Arduino, é preciso reiniciar o Arduino IDE para que seja colocado o seguinte código-fonte:

```
//Sensor de umidade e temperatura DHT11
#include <dht.h>
#define dht_dpin A1 //Pino DATA do Sensor ligado na porta
Analógica A1
```



```

dht DHT; //Inicializa o sensor

void setup()
{
  Serial.begin(9600);
  delay(1000); //Aguarda 1 seg antes de acessar as
informações do sensor
}

void loop()
{
  DHT.read11(dht_dpin); //Lê as informações do sensor
  Serial.print("Umidade = ");
  Serial.print(DHT.humidity);
  Serial.print(" % ");
  Serial.print("Temperatura = ");
  Serial.print(DHT.temperature);
  Serial.println(" Celsius ");

  //Efetuar a leitura a cada 2 segundos
  delay(2000);
}

```

Código-fonte 5 – Temperatura e Umidade
 Fonte: Elaborado pelo autor (2020)

Observe que na segunda linha do código é feito o uso de uma diretiva *include* <dht.h>, que tem o papel de literalmente incluir todo o conteúdo do arquivo em questão, fazendo com que a partir desse momento o acesso ao sensor fique transparente e por meio de funções, como `DHT.read11(dht_dpin)`, a qual efetua a leitura da umidade e temperatura, para que em seguida sejam impressas por `DHT.temperature` e `DHT.humidity`.

Para visualizar a saída pela interface da IDE, é preciso clicar em “Ferramentas”, e depois acessar “Monitor serial”, conforme a figura a seguir.

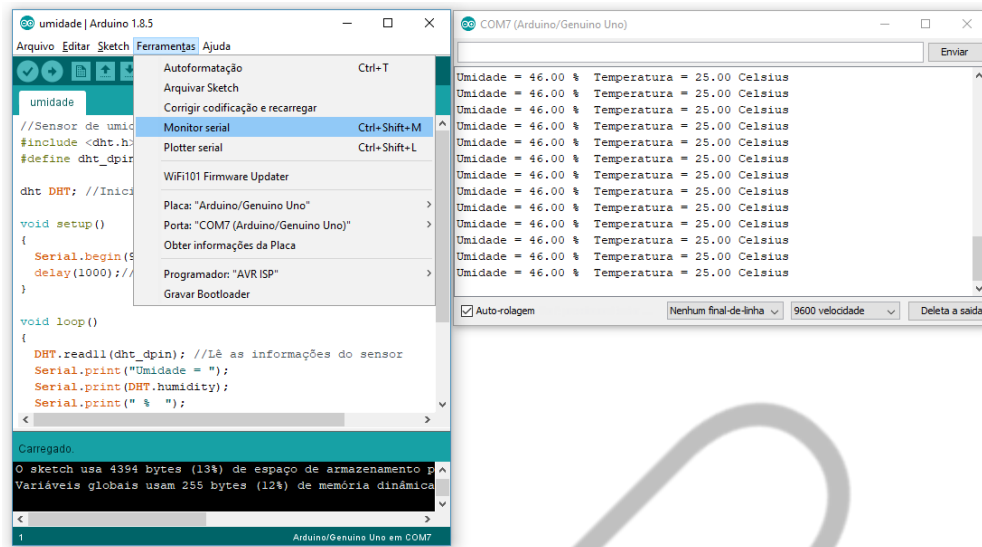


Figura 25 – Acesso ao monitor serial
Fonte: Elaborada pelo autor (2020)

11.5.2 Enviando os dados via serial

A API do Arduino possui algumas funções que efetuam a leitura dos caracteres disponíveis na porta Serial de acordo com alguma regra, dentre elas, as funções `Serial.parseInt()` e `Serial.parseFloat()` leem a porta serial em busca de um número inteiro ou ponto flutuante, e caso caracteres não numéricos sejam encontrados, eles são pulados.

Tais funções aguardam por um tempo pré-especificado (1 segundo por padrão) por mais caracteres até retornarem o valor definitivo, mas que pode ser alterado pela função `Serial.setTimeout(int milisseg)`, e caso não seja possível decodificar um número, o resultado é 0 (zero), o que pode gerar confusões no programa.

Já a função `Serial.readBytesUntil()` lê caracteres oriundos da porta serial, escrevendo em um vetor de caracteres, e termina quando o caractere de terminação for detectado, o tamanho máximo for lido ou o tempo de espera por mais texto terminou. Tal função na íntegra pode ser vista desta forma: `Serial.readBytesUntil(char terminador, char buffer[ ], int tamanho)`.

Para testar o envio de dados para o Arduino via serial, o circuito representado na Figura – “Circuito real para o pisca led” – receberá um novo código, o código-fonte abaixo, desta vez fazendo o acionamento do led pelo Monitor serial.

```
const int LED = 3;
char nextChar = 0;
void setup() {
    Serial.begin(9600);
    pinMode(LED, OUTPUT);
}
void loop() {
    if (Serial.available() > 0) {
        // lê o byte disponível na porta serial:
        nextChar = Serial.read();
        if(nextChar == 'P') {
            //Lê o próximo inteiro vindo da serial
            int valor = Serial.parseInt();
            //Atenção: em caso de erro o valor lido será 0
            analogWrite(LED, valor);
            Serial.println(String("Potencia do LED: ") +
valor);
        }
    }
}
```

Código-fonte 6 – Pisca Led
Fonte: Elaborado pelo autor (2020)

Ainda pelo código-fonte anterior, observe que somente o valor enviado pela serial deverá receber inicialmente um caracter P, seguido de um número inteiro entre 0 e 255, sendo 0 para o led desligado e 255 o led com toda a intensidade. Qualquer valor intermediário deixará o led parcialmente aceso, com uma intensidade proporcional.

Após carregar o código no Arduino, é possível fazer o acionamento do led segundo a imagem abaixo, que no caso faz o acionamento com a potência máxima do led, ao inserir o valor “P255” seguido da tecla ENTER.

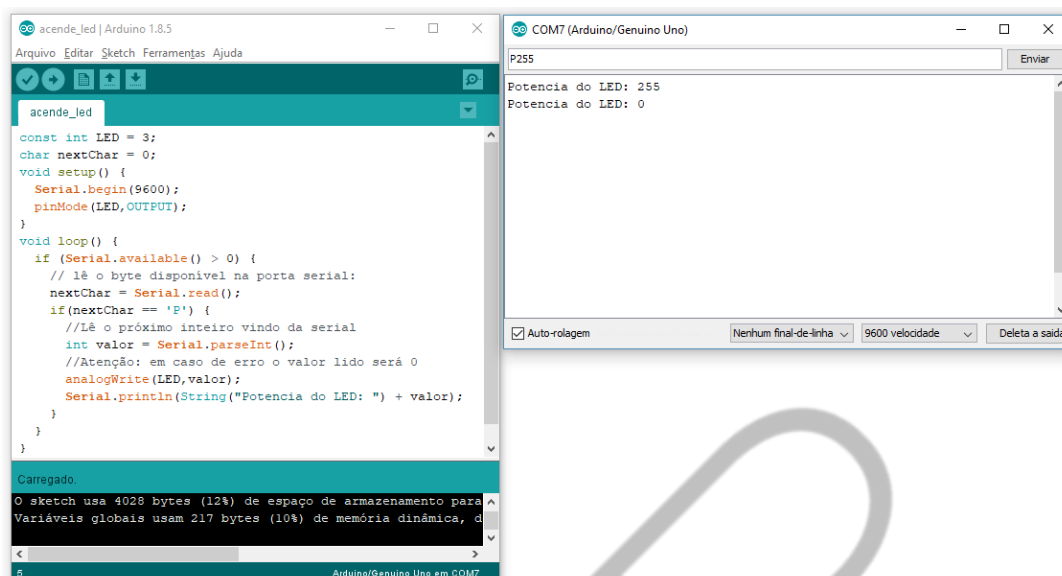


Figura 26 – Acionamento do led via serial
Fonte: Elaborada pelo autor (2020)

11.5.3 Uso de JSON

Em vez de modificar o comportamento de um atuador usando a porta serial, o mais recomendável seria ter um programa no computador que se comunicasse com o Arduino também via porta serial, e que além disso, fornecesse um serviço para que o próprio Arduino pudesse ser controlado remotamente, por algum protocolo para troca de dados de sensores e comandos para os atuadores.

Para formatar as mensagens e garantir que uma ampla gama de programas consiga se comunicar com o Arduino, o formato JSON ou *JavaScript Object Notation* entra de forma bem apropriada. De um modo geral, JSON é um formato leve de troca de dados (serialização), de fácil leitura e escrita por humanos e máquinas, e parte da especificação de 1999 do JavaScript, que codifica e decodifica JSON nativamente, somado ao fato de ser um formato de texto completamente independente de linguagem.

Na linguagem de programação C (utilizada para o Arduino) existem várias bibliotecas para a codificação e a decodificação de JSON, porém nem todas são otimizadas para rodar no Arduino.

A biblioteca que será utilizada para a construção do gateway é a [ArduinoJson](#), que relaciona objetos JSON com a estrutura de dados de dicionário

do C, que por sua vez relacionam uma chave (ou índice) a um rótulo, acrescentando dinamicamente elementos, como:

- meuDic["nome"] = "Reinaldo Belizario"; //Acrescenta um elemento.
- long valor = meuDic["idade"]; // Lê o elemento idade.

Para usar a API, a primeira providência é importar o seu cabeçalho no código, trazendo na primeira linha do programa: `#include <ArduinoJson.h>`. É importante verificar se no diretório *libraries* existe também um diretório `ArduinoJson` contendo o arquivo `ArduinoJson.h`, senão o código abaixo não será carregado.

```
#include <ArduinoJson.h>
const int LED = 3;
const int LUZ = A1;
const int TAMANHO = 200;
void setup() {
    Serial.begin(9600);
    Serial.setTimeout(10); //1000ms é muito tempo
    pinMode(LED, OUTPUT);
}
void loop() {
    if (Serial.available() > 0) {
        //Lê o texto disponível na porta serial:
        char texto[TAMANHO];
        char nextChar = 0;
        nextChar = Serial.read();
        if(nextChar == 'P') {
            //Lê o próximo inteiro vindo da serial
            int valor = Serial.parseInt();
            analogWrite(LED, valor);
        }
        Serial.readBytesUntil('\n', texto, TAMANHO);
        //Grava o texto recebido como JSON
        StaticJsonBuffer<TAMANHO> jsonBuffer;
        JsonObject& json = jsonBuffer.parseObject(texto);
        if(json.success() && json.containsKey("led")) {
            analogWrite(LED, json["led"]);
        }
    }
    StaticJsonBuffer<TAMANHO> jsonBuffer;
    JsonObject& json = jsonBuffer.createObject();
    json["luz"] = analogRead(LUZ);
    json.printTo(Serial); Serial.println();
    delay(1000);
}
```

Código-fonte 7 – Uso de JSON via porta Serial
Fonte: Elaborado pelo autor (2020)

O código acima exibirá no Monitor serial o formato JSON do valor lido através do sensor de luminosidade, e se no próprio Monitor serial for inserido P200 seguido de uma tecla ENTER, o led receberá o valor 200 e terá intensidade respectiva, chegando ao máximo de 255.

11.6 Apresentação do Gateway

A partir deste ponto, torna-se necessário utilizar o Node.js mencionado no início do capítulo, um ambiente de execução em tempo real para a linguagem JavaScript, com foco na execução de programas no lado do servidor.

Para instalar o Node.js no sistema operacional Windows, a tática adotada será a de utilizar uma versão portátil, pois facilita sobremaneira ter o servidor funcionando rapidamente, e para isto, basta fazer o download em: <https://sourceforge.net/projects/nodejsportable/>.

A instalação é simples, e recomenda-se o diretório C:\nodejs. E dentro deste diretório existirá o arquivo executável NodeJSPortable.exe, bastando clicar com o botão esquerdo que abrirá a linha de comando, como se fosse o MSDOS tradicional, e se tudo estiver perfeito a figura abaixo será identificada visualmente da seguinte forma:

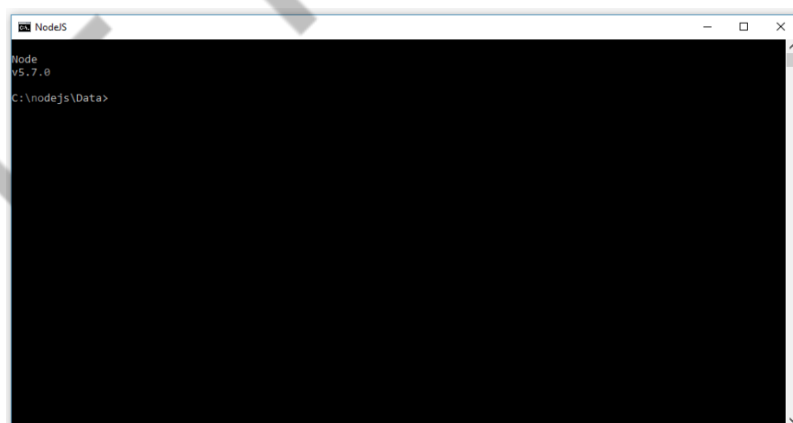


Figura 27 – Linha de comando do Node.js
Fonte: Elaborada pelo autor (2020)

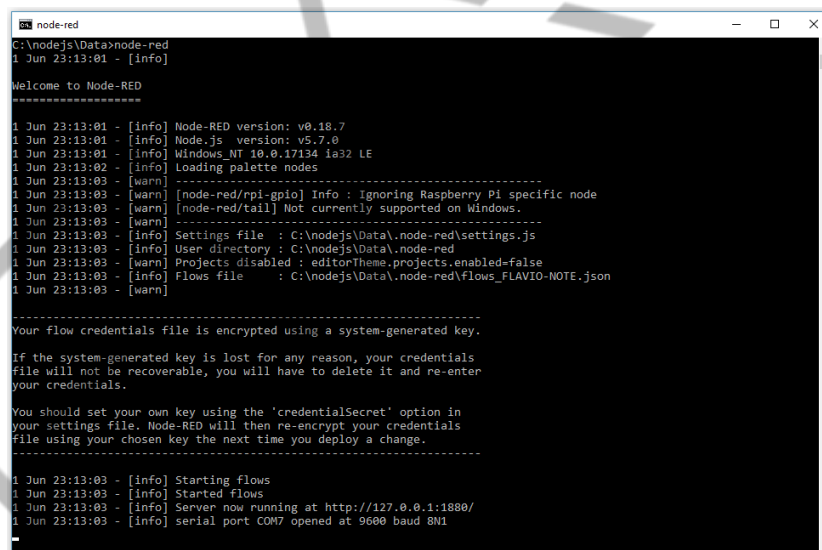
Uma vez que a programação do Node.js é assíncrona, é considerável pensar em programas em que todas as ações são acionadas por um evento gatilho. Sendo assim, também é comum pensar na programação do Node.js como fluxos de dados

que iniciam a partir de algum evento, como o disparo de um temporizador, a requisição de um cliente de webservice ou um dado vindo do Arduino.

Nesse momento, o Node-Red surge para encurtar o caminho, pois é um serviço escrito para Node.js que provê uma ferramenta visual para editar fluxos de mensagens, vindas de diferentes fontes, podendo ser processadas e mandadas para diferentes destinos, como uma conta de e-mail ou do X.

De forma bem simplificada, o Node-Red tem uma ferramenta para edição dos fluxos que roda no próprio browser, sendo possível exportar e importar fluxos no formato JSON usando o menu de opções. E para instalá-lo, é preciso, dentro do prompt visto pela figura anterior, digitar: `npm install -g --unsafe-perm node-red`, e para subir o serviço na sequência é preciso digitar `node-red`.

Se tudo estiver correto, o resultado apresentado será equivalente ao reportado pela imagem a seguir.



```

node-red
C:\nodejs\Data>node-red
1 Jun 23:13:01 - [info]
Welcome to Node-RED
-----
1 Jun 23:13:01 - [info] Node-RED version: v0.18.7
1 Jun 23:13:01 - [info] Node.js version: v5.7.0
1 Jun 23:13:01 - [info] Windows_NT 10.0.17134 ia32 LE
1 Jun 23:13:02 - [info] Loading palette nodes
1 Jun 23:13:03 - [warn]
1 Jun 23:13:03 - [warn] [node-red/rpi-gpio] Info : Ignoring Raspberry Pi specific node
1 Jun 23:13:03 - [warn] [node-red/tail] Not currently supported on Windows.
1 Jun 23:13:03 - [warn]
1 Jun 23:13:03 - [info] Settings file : C:\nodejs\Data\node-red\settings.js
1 Jun 23:13:03 - [info] User directory : C:\nodejs\Data\node-red
1 Jun 23:13:03 - [warn] Projects disabled : editorTheme.projects.enabled=false
1 Jun 23:13:03 - [info] Flows file : C:\nodejs\Data\node-red\flows_FLAVIO-NOTE.json
1 Jun 23:13:03 - [warn]
-----
Your flow credentials file is encrypted using a system-generated key.
If the system-generated key is lost for any reason, your credentials
file will not be recoverable, you will have to delete it and re-enter
your credentials.
You should set your own key using the 'credentialSecret' option in
your settings file. Node-RED will then re-encrypt your credentials
file using your chosen key the next time you deploy a change.
-----
1 Jun 23:13:03 - [info] Starting flows
1 Jun 23:13:03 - [info] Started flows
1 Jun 23:13:03 - [info] Server now running at http://127.0.0.1:1880/
1 Jun 23:13:03 - [info] serial port COM7 opened at 9600 baud 8N1
  
```

Figura 28 – Subindo o serviço do Node-Red

Fonte: Elaborada pelo autor (2020)

Todo o procedimento, a partir do momento em que o servidor Ned-Red estiver funcionando, será realizado pelo navegador, então o endereço <http://localhost:1880> significa atingir o resultado visual idêntico ao da imagem abaixo.

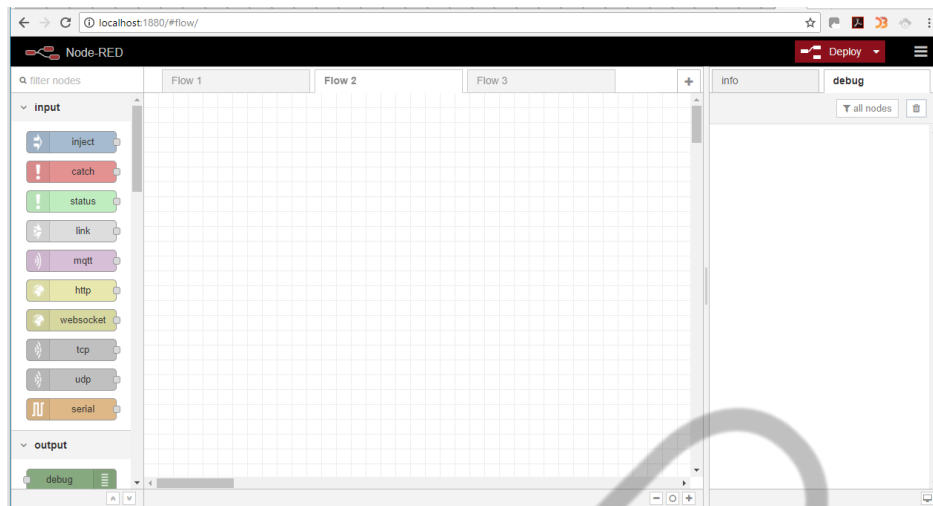


Figura 29 – Tela inicial do Node-Red
Fonte: Elaborada pelo autor (2020)

11.6.1 Sobre os nós do Node-Red

De um modo geral, existem basicamente três tipos de “nodes”:

- **Entrada:** que emitem mensagens a partir de eventos de entrada.
- **Processamento:** que converte mensagens de entrada em mensagens de saída.
- **Saída:** que envia a mensagem de entrada para ser consumida em algum lugar.

Dentre os nós de processamento, podemos também criar funções JavaScript genéricas que podem manipular os campos da mensagem como se desejar, como a conversão de um timestamp em uma data pré-formatada:

```
function(msg) {
  msg.payload = new Date(msg.payload).toString();
  return msg;
}
```

Agora, para usar as funcionalidades da porta serial, é necessário instalar o pacote do Node.js chamado **node-red-node-serialport**, do seguinte modo na linha comando: `npm install -g --unsafe-perm node-red-node-serialport`

A captura da porta serial completa a leitura quando:

- recebe um caractere de terminação (default: '\n') ou

- estoura o tempo limite (timeout) para novos caracteres ou
- estoura o número máximo de caracteres que podem ser lidos.

O node de saída para a porta serial envia apenas o campo **msg.payload**, podendo ser configurado um caractere de terminação, e quando o Arduino é a fonte de dados, é representado no Node-Red como um *node* de **entrada** do tipo *serial port* (a **saída** de dados do Arduino é usada como **entrada** de dados pelo Node-Red).

Na configuração da porta serial, fundamentalmente, dois itens devem ser considerados, conforme também mostra a figura abaixo:

- O nome da porta serial deve ser o mesmo onde o Arduino está conectado.
- A velocidade de comunicação deve ser a mesma da do Arduino, geralmente 9600 *baud*.
- Caso o Arduino envie quebras de linhas após cada dado enviado, podemos deixar que a porta serial dispare a cada vez que encontrar o caractere `\n`.

Os próximos exemplos serão viáveis quando o código-fonte abaixo for carregado no Arduino. Observe que há muita proximidade com o código visto anteriormente.

```
#include <ArduinoJson.h>
const int LED = 3;
const int LUZ = A1;
const int TAMANHO = 200;
void setup() {
  Serial.begin(9600);
  Serial.setTimeout(10); //1000ms é muito tempo
  pinMode(LED, OUTPUT);
}
void loop() {
  if (Serial.available() > 0) {
    //Lê o texto disponível na porta serial:
    char texto[TAMANHO];
    char nextChar = 0;
    nextChar = Serial.read();
    if(nextChar == 'P') {
      //Lê o próximo inteiro vindo da serial
      int valor = Serial.parseInt();
      analogWrite(LED, valor);
    }
    Serial.readBytesUntil('\n', texto, TAMANHO);
    //Grava o texto recebido como JSON
    StaticJsonBuffer<TAMANHO> jsonBuffer;
```

```
JsonObject& json = jsonBuffer.parseObject(texto);
if(json.success() && json.containsKey("led")) {
    analogWrite(LED, json["led"]);
}
}
StaticJsonBuffer<TAMANHO> jsonBuffer;
JsonObject& json = jsonBuffer.createObject();
json["luz"] = analogRead(LUZ);
json.printTo(Serial); Serial.println();
delay(1000);
}
```

Código-fonte 8 – Controle de led e luminosidade via Node-Red

Fonte: Elaborado pelo autor (2020)

Novamente citando o detalhamento do node serial, é possível visualizar pela figura a seguir quais as opções viáveis pelo Node-Red:

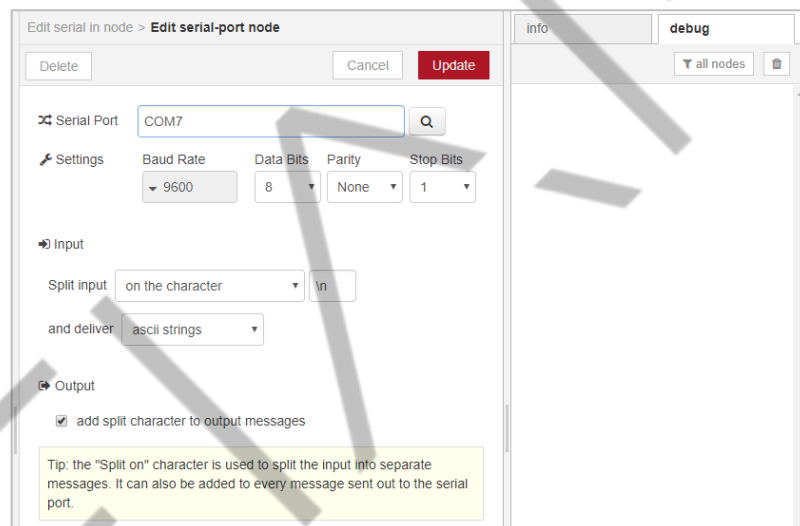


Figura 30 – Configuração da porta serial no Node-Red

Fonte: Elaborada pelo autor (2020)

E como ficam as trocas de dados com o Arduino? A forma mais simples de comunicação do Node-Red com o Arduino é injetando diretamente os comandos JSON através do node “inject”, e verificar diretamente a saída com o node “debug”, como mostramos a seguir:

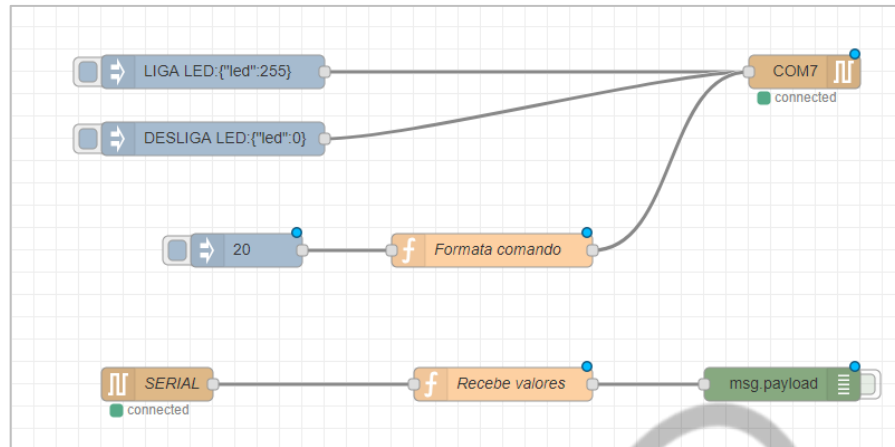


Figura 31 – Exemplo de fluxo usando Node-Red
Fonte: Elaborada pelo autor (2020)

Com o objetivo de deixar mais claro como é feito o controle de um led da placa, pode ser visto por meio da imagem que se segue, na qual o node do tipo inject é detalhado, mostrando como o valor no formato JSON deve ser inserido.

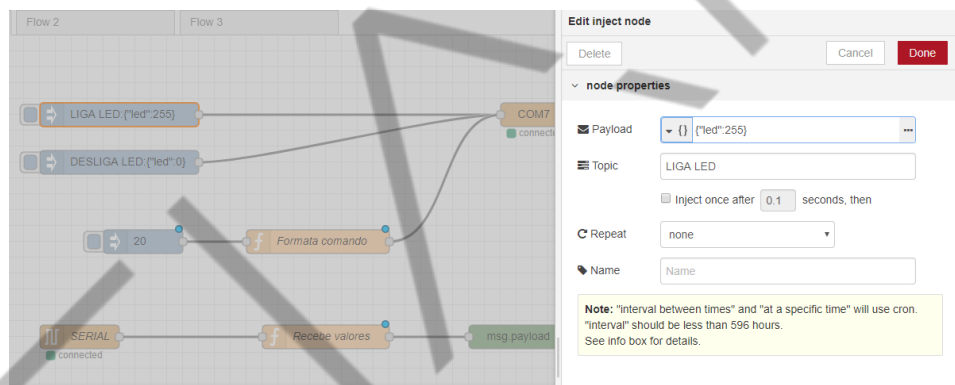


Figura 32 – Detalhamento do nó inject do Node-Red
Fonte: Elaborada pelo autor (2020)

Se por um lado o envio de dados para a placa está ilustrado, de outro a figura a seguir explicita o modo como os dados são recebidos por ela, e através de um *node debug* o valor pode ser exibido no lado direito do navegador, ao deixar o *módulo debug* habilitado.

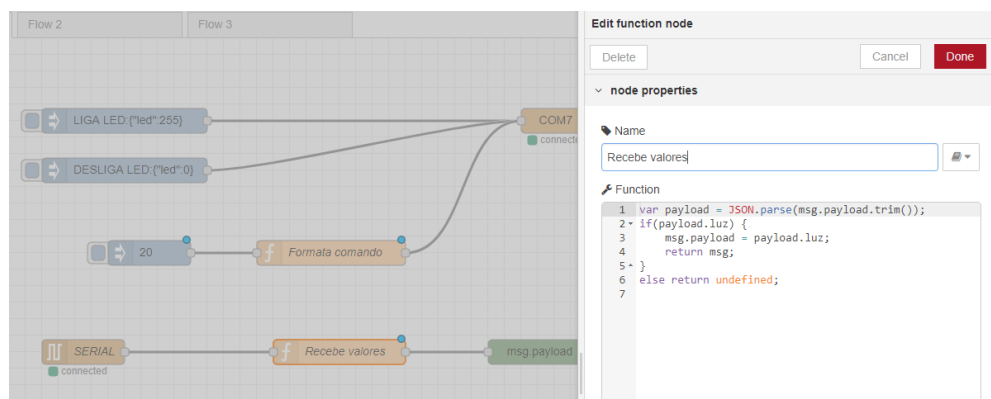
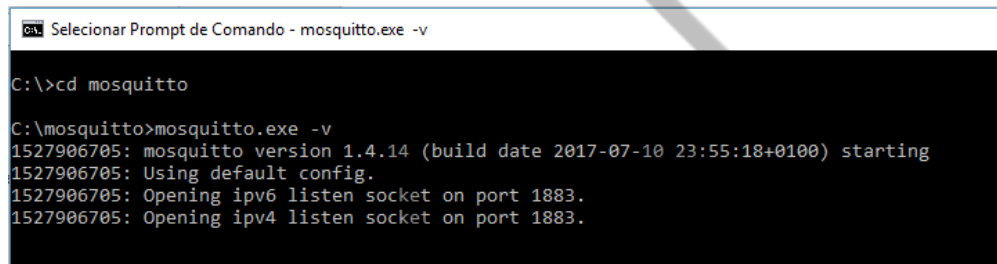


Figura 33 – Recebimento de dados da placa via Node-Red
Fonte: Elaborada pelo autor (2020)

11.6.2 Usando o broker Mosquitto

O objetivo neste ponto é mostrar como implementar a troca de mensagens usando o protocolo MQTT, e para isso, é importante ter um *broker* ou MQTT server funcionando localmente. Para que seja factível, é preciso fazer o download do arquivo em <http://www.steves-internet-guide.com/wp-content/uploads/mos1.14.7z> e descompactar o arquivo no diretório c:\mosquitto.

Usando o prompt de comando do MSDOS, e acessando a pasta c:\mosquitto, é fácil rodar o serviço, digitando: `mosquitto -v`, de acordo com a imagem abaixo.



```

Selecionar Prompt de Comando - mosquitto.exe -v

C:\>cd mosquitto

C:\mosquitto>mosquitto.exe -v
1527906705: mosquitto version 1.4.14 (build date 2017-07-10 23:55:18+0100) starting
1527906705: Using default config.
1527906705: Opening ipv6 listen socket on port 1883.
1527906705: Opening ipv4 listen socket on port 1883.
    
```

Figura 34 – Subindo localmente o MQTT broker Mosquitto
Fonte: Elaborada pelo autor (2020)

O ponto final para este conteúdo é mostrar como fazer a troca de mensagens usando o *broker*, que por sinal utiliza a porta 1883, também visto na figura acima.

E como criar o fluxo relativo ao Gateway? Por meio de dois tópicos envolvendo a publicação e subscrição:

- Publica as leituras do sensor de luminosidade na forma numérica (0 a 1023):
fiap/mqtt/luz
- Subscrive comandos numéricos para definir o brilho do LED (0 a 255):
fiap/mqtt/led

O fluxo Node-Red relativo ao Gateway pode ser observado na figura, a seguir, evidenciando o status “connected” indicando que o servidor MQTT está funcionando corretamente.

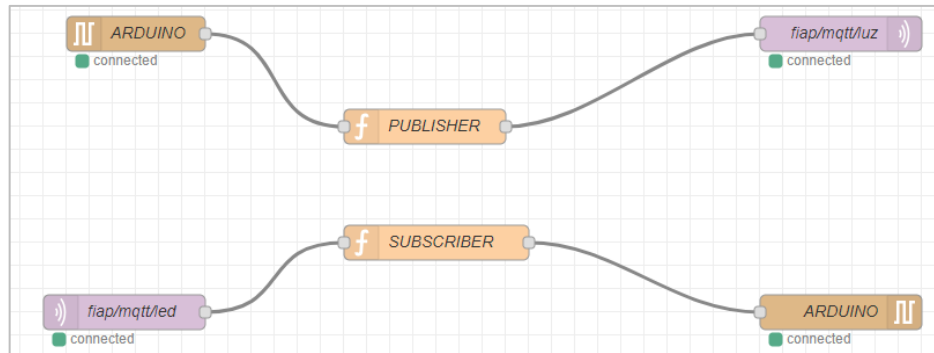


Figura 35 – Fluxo Node-Red mostrando os tópicos
Fonte: Elaborada pelo autor (2020)

E quanto aos nodes do tipo MQTT, como configurar? Neste caso é bastante simples, e pode ser observado em conjunto na imagem abaixo, já com as configurações locais do Mosquitto Broker.

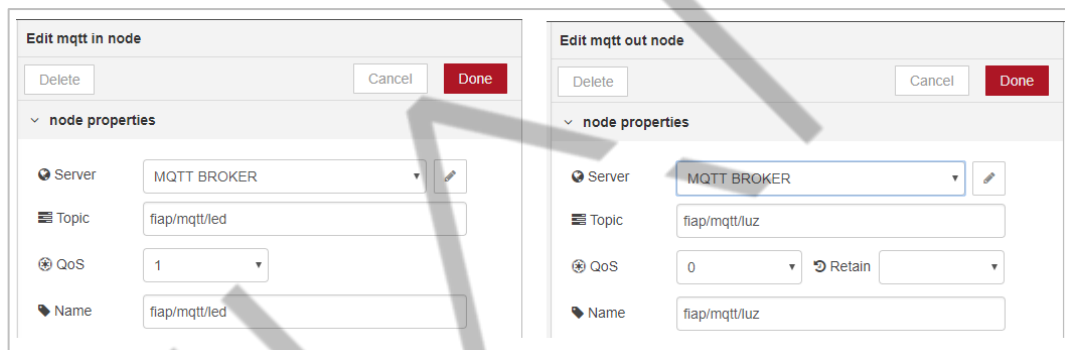


Figura 36 – Configuração dos nodes MQTT via Node-Red
Fonte: Elaborada pelo autor (2020)

E agora, como é possível testar o fluxo? Também é bastante simples, e para isso precisa de um fluxo novo, como pode ser visto abaixo.

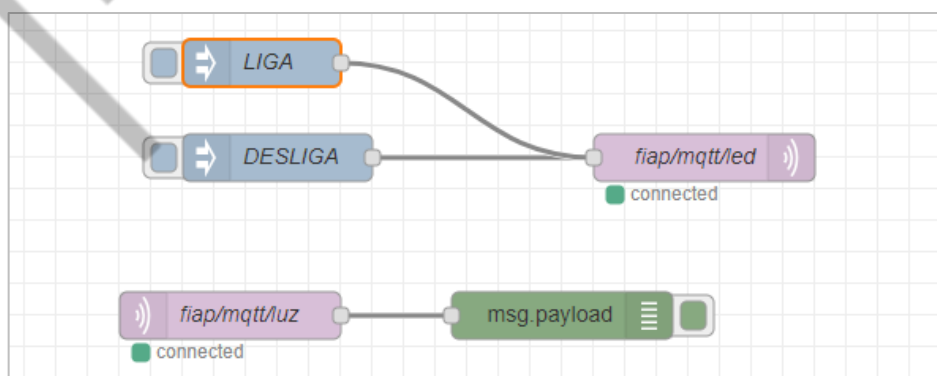


Figura 37 – Fluxo para teste do Gateway via Node-Red
Fonte: Elaborada pelo autor (2020)

Por meio da utilização da aba “debug” do editor de fluxo do Node-Red, é possível exibir os envios feitos pelo PUBLISHER e, ao mesmo tempo, é possível

acionar o acendimento do led por meio do SUBSCRIBER. A parte do PUBLISHER é materializada pela imagem a seguir.

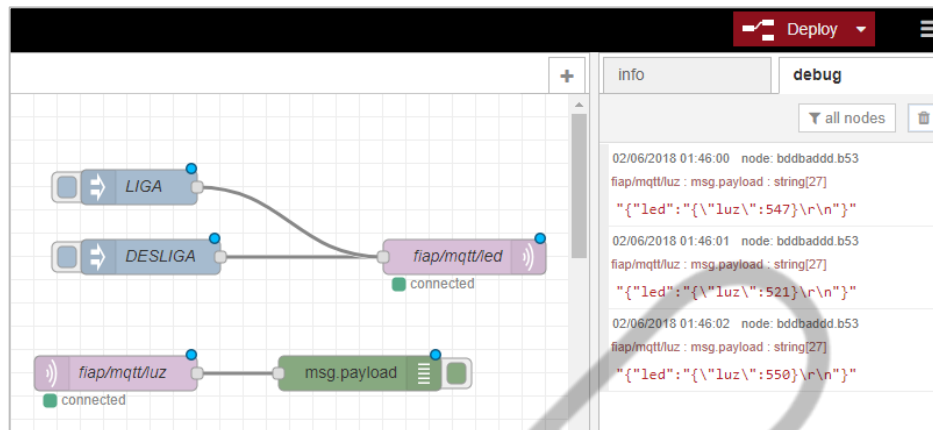


Figura 38 – Aba “debug” do editor de fluxo do Node-Red
Fonte: Elaborada pelo autor (2020)

12 CONCATENANDO AS IDEIAS APRESENTADAS

Diante de uma avalanche de informações, foi possível mostrar como todo o processo de envio e recebimento de dados envolvendo “dispositivos conectados” funciona, desde a programação no Arduino (inclusive com o uso da plataforma TinkerCad), utilização da porta serial e criação de um gateway para tornar o processo mais automatizado e não dependente somente da máquina em que o Arduino está diretamente conectado.

Que venha o próximo desafio: utilizar uma plataforma corporativa para compreender que, até o momento, foi possível construir algo artesanal, ficando restrito o escalonamento, alta disponibilidade e confiabilidade – questões que estão presentes em soluções comerciais.

REFERÊNCIAS

ARDUINO. **Modelo Arduino UNO**. 2020. Disponível em: < [Arduino - Home](#)>. Acesso em: 10 abr. 2020.

ARDUINO IDE. **Download the Arduino IDE**. 2020. Disponível em: <<https://www.arduino.cc/en/Main/Software>>. Acesso em: 10 abr. 2020.

DHT. **Download da biblioteca DHT**. 2020. Disponível em: <<http://hobbyist.co.nz/sites/default/files/WeatherStation/DHT.zip>>. Acesso em: 2 mai. 2020.

DREAMINC. **Sobre a protoboard**. 2020. Disponível em: <http://www.dreaminc.com.br/sala_de_aula/protoboard/>. Acesso em: 15 abr. 2020.

GARDUINO. **Aplicação simples em jardins**. 2020. Disponível em: <<http://www.instructables.com/id/Garduino-Gardening-Arduino>>. Acesso em: 15 abr. 2020.

IOT LEARNERS. **Learn IoT Gateways and how to use them**. 2020. Disponível em: <<https://iotlearners.com/learn-iot-gateways-and-how-to-use-them>>. Acesso em: 15 abr. 2020.

JSON ARDUINO. **Biblioteca Json para Arduino**. 2020. Disponível em: <<https://github.com/bblanchon/ArduinoJson>>. Acesso em: 4 mai. 2020.

MOSQUITTO. **Instalador automático do Mosquitto Broker**. 2020. Disponível em: <<http://www.steves-internet-guide.com/wp-content/uploads/mos1.14.7z>>. Acesso em: 15 abr. 2020.

NODE PORTABLE. **Versão portable do Node.js**. 2020. Disponível em: <<https://sourceforge.net/projects/nodejsportable/>>. Acesso em: 10 abr. 2020.

RASPBERRY. **Modelo Raspberry-Pi B+**. 2020. Disponível em: <<https://www.raspberrypi.org/products/raspberry-pi-3-model-b-plus>>. Acesso em: 15 abr. 2020.